KISVÁRDAI SZC FEHÉRGYARMATI PETŐFI SÁNDOR TECHNIKUM
4900 Fehérgyarmat, Május 14. tér 16.
203043/003

Szoftverfejlesztő és - tesztelő technikus képzés
5 0613 12 03

VIZSGAREMEK

HistóriaWeb

Térkép alapú történetmegosztó platform

Konzulens:
Lakatos Sándor
Berki Balázs

Készítette:
Tivadar Dávid

FEHÉRGYARMAT

2026

KISVÁRDAI
VIZSGAKÖZPONT

4600 Kisvárda, Mártírok útja 8.
45/556-391
kisvardai.vizsgakozpont@gmail.com
www.kisvardaivizsgakozpont.hu
Akkreditált szakképzési vizsgaközpont

Vizsgaportfólió szabályzat 2.sz. melléklete

Nyilatkozat

Alulírott Tivadar Dávid, Jéhegyalmat, 2006. 10.07. (név, születési hely és idő) jelen nyilatkozat aláírásával kijelentem, hogy a 5-0613-12-03 szoftverfejlesztő és -tesztelő szakmai vizsgára készített portfólió saját munkám eredménye.

Nyilatkozom, hogy az egyes dokumentumokban foglalt tények és adatok a megadott forrásmunkák felhasználásán alapulnak.

Tudomásul veszem, hogy amennyiben a portfóliómban a vizsgabizottság tagjai olyan dokumentumot találnak, amely problémaorientált feladat esetén 20%-nál nagyobb mértékben nem önálló munkám eredménye, vagy leíró/összegző feladat esetén nem saját kutatásom eredménye és amelyből a források kiválasztásnak gondolatmenete vagy relevanciája nem követhető, valamint nem tartalmazza a pontos forrásmegjelöléseket, abban az esetben a vizsgabizottság tagjai a portfóliót nem fogadják el. Ezen pont alól kivételt képeznek a csoportos projektek, komplex feladatok és esettanulmányok, amelyek keletkezési körülményeit a bevezető tartalmazza.

Jelen nyilatkozat aláírásával tudomásul veszem, hogy amennyiben bizonyítható, hogy a portfóliót nem magam készítettem, a prezentáció nem kezdhető meg, így ez a vizsgarész elégtelen értékelést kap.

Hozzájárulok ahhoz, hogy vizsgabizottság tagjai, jegyzője, és a *Kisvárdai Vizsgaközpont* munkatársai a portfólió tartalmát megismerhessék.

Kijelentem, hogy a portfólióra kapott értékelésemet és a portfóliómat további felhasználásra sem kinyomtatva, sem pedig elektronikus úton nem továbbítom senkinek.

Kelt: 2026. március 13. Jéhegyalmat

Tivadar Dávid
aláírás

KISVÁRDAI SZC FEHÉRGYARMATI PETŐFI SÁNDOR TECHNIKUM
4900 Fehérgyarmat, Május 14. tér 16.
203043/003
Szoftverfejlesztő és - tesztelő technikus képzés

Eredetiségi nyilatkozat projektmunkáról

Alulírott Fivadar David 72785957916 (saját név, oktatási azonosító) nyilatkozom, hogy a csatoltan bírálatra és védésre beadott projektmunka teljes egészében a saját és ~~két~~ csapattársam – Kozma Patrik 72826127866 névl,(oktatási azonosítól), ……névl (okt.azon2)- munkája.

A felhasznált forrásokat az irodalomjegyzékben feltüntettem, a rájuk vonatkozó szabályszerű hivatkozásokat a szövegben megtettem. A projektmunka más intézményben sem a saját nevemben, sem más nevében nem került beadásra.

Tudatában vagyok annak, hogy plágium (más munkájának sajátomként történő feltüntetése) esetén a projektmunka érvénytelen, ezért elutasításra kerül.

Fehérgyarmat, 2026. március 13.

Fivadar David
aláírás

TARTALOMJEGYZÉK

Bevezetés 3

1. A Frontend Működése 4

1.1. Mi a frontend és miért fontos? 4

1.2. A főoldal (Fooldal komponens) 4

1.3. A hitelesítő oldal (Hitelesito komponens) 5

1.4. Történetek böngészése település szerint 6

1.5. Egyedi történet megjelenítése 7

1.6. Új történet írása 8

1.7. Saját történet szerkesztése 9

1.8. Összes történet böngészése 9

1.9. Fiók szerkesztése 10

1.10. Navigációs sáv (Navbar komponens) 10

1.11. Lábléc (Footer komponens) 11

2. A Backend Működése 11

2.1. Mi a backend és miért fontos? 11

2.2. Az adatbázis felépítése 11

2.3. API végpontok (Endpoints) 12

2.4. Jogosultságok ellenőrzése 16

2.5. Adatbázis kapcsolat kezelése 16

3. Használt Technológiák és Eszközök 16

3.1. Angular keretrendszer 16

3.2. TypeScript programozási nyelv 19

3.3. Node.js backend környezet 21

3.4. Express.js keretrendszer 22

3.5. MySQL adatbázis 23

3.6. Bootstrap CSS keretrendszer 25

3.7. GitHub – verziókezelés 26

3.8. React Native mobilalkalmazás 27
3.9. Tesztelés – Jasmine és Jest 29
3.10. Visual Studio Code (VS Code) 31
3.11. XAMPP fejlesztői környezet 32
3.12. Figma – UI/UX design 33
3.13. Projektmenedzsment eszközök 34
3.14. Külső források és adatok 34
3.15. Összefoglalás – Technológiai stack 35
4. Munkamegosztás 36
4.1. Kozma Patrik – Backend 36
4.2. Tivadar Dávid – Frontend 36
4.3. Fejlesztési módszertan és sprintek 37
5. Kihívások és Megoldások 38
6. Statisztikai Adatok és Eredmények 39
7. SWOT Analízis 42
8. Továbbfejlesztési Lehetőségek 47
8.1. AI integráció – Magyar BERT modell 47
9. Mit Adott Nekünk Ez a Projekt 49
10. Mit Várunk el a Projekttől 51
11. Köszönetnyilvánítás 53
Összefoglalás 54
Irodalomjegyzék 55
Ábrajegyzék 56

BEVEZETÉS

A projekt célja és háttere

A HistóriaWeb egy közösségi oldal, ahol a felhasználók Magyarország településeihez kapcsolódó történeteket osztanak meg. Az oldal célja, hogy az emberek felfedezzék a helyi történeteket, legendákat és kulturális érdekességeket.

A digitális korban fontos, hogy támogassuk a helyi közösségeket és megőrizzük a kulturális örökséget. A HistóriaWeb nem egy szokásos tartalommegosztó oldal, hanem kifejezetten a magyar települések egyedi történeteinek összegyűjtésére és digitális megőrzésére jött létre.

A weboldal nem csak olvasásra szolgál, hanem aktív közösségi részvételre is ösztönöz. A felhasználók megoszthatják saját történeteiket, értékelhetik mások bejegyzéseit, és hozzászólásokat is írhatnak.

A projekt indokoltsága

Magyarországon jelenleg nincs olyan platform, amely kifejezetten a települések történeteinek gyűjtésére szolgál. A projekt kulturális és oktatási értéket is ad, mert segít megőrizni olyan helyi tudást, amely máskülönben elveszne.

A platform hasznos lehet turistáknak is, akik helyiek által megosztott információkat keresnek. Tanárok, helytörténészek és kutató diákok is értékes forrásanyagot találhatnak itt. A rendszer nemcsak a múlt történeteinek megosztására jó, hanem a jelen eseményeinek dokumentálására is, így folyamatosan frissülő kulturális térkép jön létre.

A dokumentáció felépítése

Ez a dokumentáció részletesen bemutatja, hogyan működik a HistóriaWeb. Először a felhasználói felületet, vagyis a frontend részt ismertetjük, és végigvezetjük az egyes funkciókat és oldalakat. Megmutatjuk, hogyan navigálhat a felhasználó, milyen műveleteket végezhet, és ezek hogyan jelennek meg a képernyőn.

Ezután bemutatjuk a szerveroldali működést, vagyis a backend részt, ahol látható, hogyan tárolódnak az adatok, hogyan kommunikál a frontend a backenddel, és hogyan védi a rendszer a felhasználói információkat. A végén kitérünk a jövőbeli fejlesztési lehetőségekre is, amelyek tovább bővíthetik a platform funkcionalitását és felhasználói élményét.

1. A FRONTEND MŰKÖDÉSE

1.1 Mi a frontend és miért fontos?

1. ábra

A frontend az, amit a felhasználók látnak és amivel kapcsolatba lépnek. Ide tartozik az oldal kinézete, a gombok, szövegek, képek és minden elem.

A HistóriaWeb-nél a frontend gondoskodik arról, hogy az oldal jól nézzen ki és könnyen használható legyen. Minden funkció egyszerűen elérhető el, és az oldalnak minden eszközön megfelelően néznek ki, akár számítógépen, tableten vagy okostelefonon használják.

A frontend fejlesztéséhez az Angular nevű keretrendszert használjuk. Ez segít abban, hogy modern weboldalakat könnyebben építsünk. Az Angular lehetővé teszi, hogy az oldal gyors legyen és ne kelljen minden kattintásnál újratölteni az oldalt.

1.2 A főoldal (Fooldal komponens)

Működés

A főoldal az első, amit a látogatók látnak, amikor megnyitják a HistóriaWeb-et. Ez az üdvözlő oldal, ahol bemutatjuk az oldal célját és fő funkcióit. Itt található Magyarország térképe, ami a főoldal központi része.

Minden olyan település jelölve van, amelyhez már tartozik történet. Ha az egeret egy település fölé viszed, megjelenik a neve. Ha rákattintasz, az adott település oldalára kerülsz, ahol elolvashatod a hozzá tartozó történeteket.

Megvalósítás részletei

A térkép adatait a backend szolgáltatja, és folyamatosan frissíti azokat a településeket, amelyekhez új történetek kerülnek fel.

A főoldal tetején található egy navigációs sáv, amely mindenhol ugyanaz marad az oldalon. Ez tartalmazza a főbb menüpontokat: Főoldal link, ami bármikor visszahoz erre az oldalra; Összes történet link, amely megjeleníti az összes feltöltött történetet; Történet megosztás link (csak bejelentkezett felhasználóknak), ahol új történetet lehet írni; Fiók szerkesztés link (csak bejelentkezett felhasználóknak), ahol a fiókadatait lehet módosítani; Bejelentkezés/Regisztráció link (nem bejelentkezett felhasználóknak); Kijelentkezés link (bejelentkezett felhasználóknak)

A főoldal alján van egy lábléc, amely minden aloldalon ugyanúgy megjelenik. Itt vannak egyéb fontos információk, elérhetőségek.

1.3 A hitelesítő oldal (Hitelesito komponens)

2. ábra

Működés

A hitelesítő oldal felelős a bejelentkezésért és regisztrációért. Ez az oldal akkor jelenik meg, amikor a felhasználó a “Bejelentkezés” gombra kattint.

Az oldalon két különböző űrlap található. Az első a bejelentkezési űrlap, ahol a már regisztrált felhasználók megadhatják a felhasználónevüket és jelszavukat. A második a regisztrációs űrlap, ahol az új felhasználók létrehozhatják a fiókjukat.

Ha bejelentkezésnél a megadott adatok helyesek, az oldal automatikusan átirányít a főoldalra, és onnantól bejelentkezett felhasználóként böngészhet.

A regisztrációs felületen meg kell adni egy választott felhasználónevet, egy email címet és egy jelszót. A rendszer ellenőrzi, hogy a megadott felhasználónév még nem foglalt-e, és hogy a jelszó elég erős-e. Ha minden rendben van, létrejön az új fiók, és a felhasználó automatikusan be is jelentkezik.

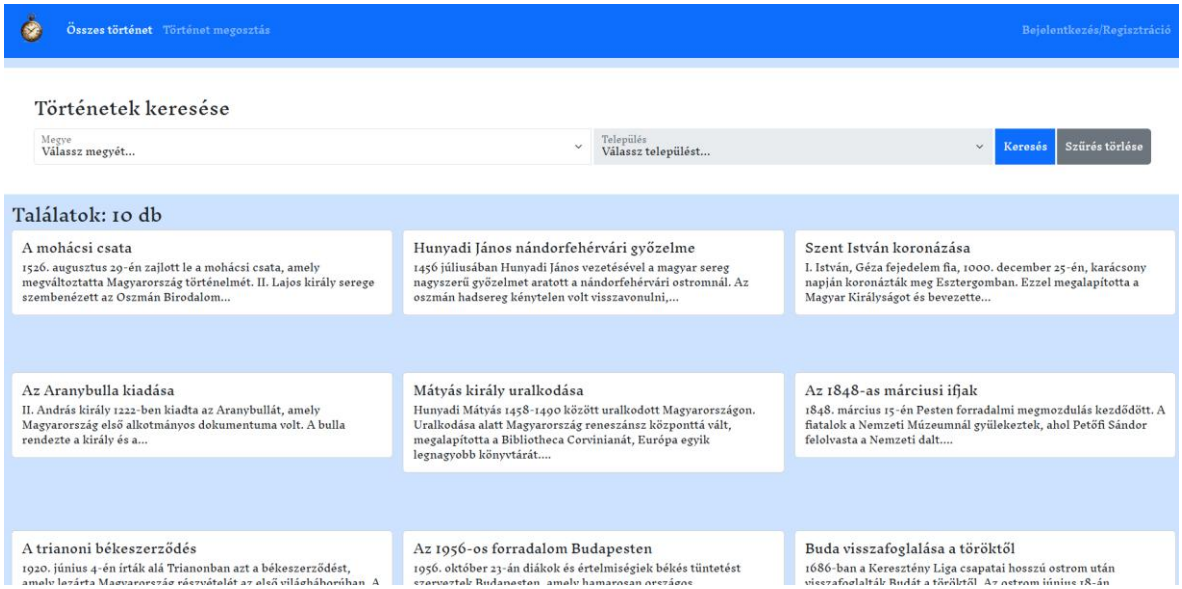
Megvalósítás részletei

A bejelentkezési folyamat során a frontend összegyűjti az űrlapon megadott adatokat. Ezután elküldi ezeket a backend-nek, amely ellenőrzi, hogy létezik-e ilyen felhasználó az adatbázisban, és hogy a jelszó helyes-e. Ha minden rendben van, a backend visszaküld egy megerősítést és a felhasználó adatait elmenti a böngésző memóriájába. Ez azért hasznos, mert a felhasználónak nem kell minden oldalbetöltésnél újra bejelentkeznie.

A regisztráció során hasonló folyamat zajlik le, azzal a különbséggel, hogy itt új felhasználót hozunk létre az adatbázisban. A frontend ellenőrzi, hogy: a felhasználónév nem üres, az email cím valódi email formátumban van megadva, a jelszó elég hosszú és biztonságos (minimum 8 karakter, kis- és nagybetű, szám és speciális karakter szükséges), a jelszó és a megerősítő mező egyezik, és a megadott felhasználónév még nincs foglalva.

Ha bármelyik ellenőrzés sikertelen, akkor hibaüzenet jelenik meg az adott mező mellett, amely pontosan megmondja, mi a probléma.

1.4 Történetek böngészése település szerint (Tortenetek komponens)



3. ábra

Működés

Amikor a felhasználó a térképen rákattint egy településre, vagy beír egy településnevet a keresőbe, akkor megjelenik a település oldala. Ez az oldal listázza az összes történetet, amely az adott településhez tartozik.

Minden történet egy különálló kártyán jelenik meg. A kártyán látható a történet címe és egy rövid előnézet a szövegéből. Amikor a felhasználó rákattint egy történetre, akkor megnyílik a történet teljes oldala, ahol elolvashatja a teljes szöveget, értékelheti azt és hozzászólhat. A történetek listája automatikusan frissül, amikor új történetek kerülnek fel az adott településhez. A legújabb történetek jelennek meg elöl, így a felhasználók mindig először az

újdonságokat látják.

Megvalósítás részletei

Amikor a felhasználó meglátogat egy település oldalt, a frontend elküldi a település nevét a backend-nek. A backend az adatbázisból kikeresi az összes olyan történetet, amely ehhez a településhez tartozik, és visszaküldi azokat a frontend-nek.

A frontend ezután megjeleníti ezeket a történeteket. Minden történet egy kártyaszerű dobozban jelenik meg, amely tartalmazza: a történet címét nagy betűkkel; a történet kis részletét; a létrehozás dátumát. A kártyák rácsban jelennek meg, rendezett sorokban.

A kártyák rácsban jelennek meg, rendezett sorokban. Mobilon egy oszlopban, számítógépen pedig több oszlopban láthatók.

Ha a felhasználó rákattint egy kártyára, akkor átirányítódik a történet részletes oldalára. Itt már a teljes szöveget olvashatja, láthatja a szerző nevét, és lehetősége van értékelni vagy hozzászólni.

1.5 Egyedi történet megjelenítése (Tortenet komponens)

Működés

Ez az oldal akkor jelenik meg, amikor a felhasználó rákattint egy konkrét történetre. Itt teljes részletességgel látható a kiválasztott történet minden információja.

Az oldal tetején megjelenik a történet címe nagy betűkkel. Az oldalon szerepel a szerző felhasználóneve és a történet feltöltésének dátuma. Ha a történethez kép is tartozik, az a szöveg alatt meg.

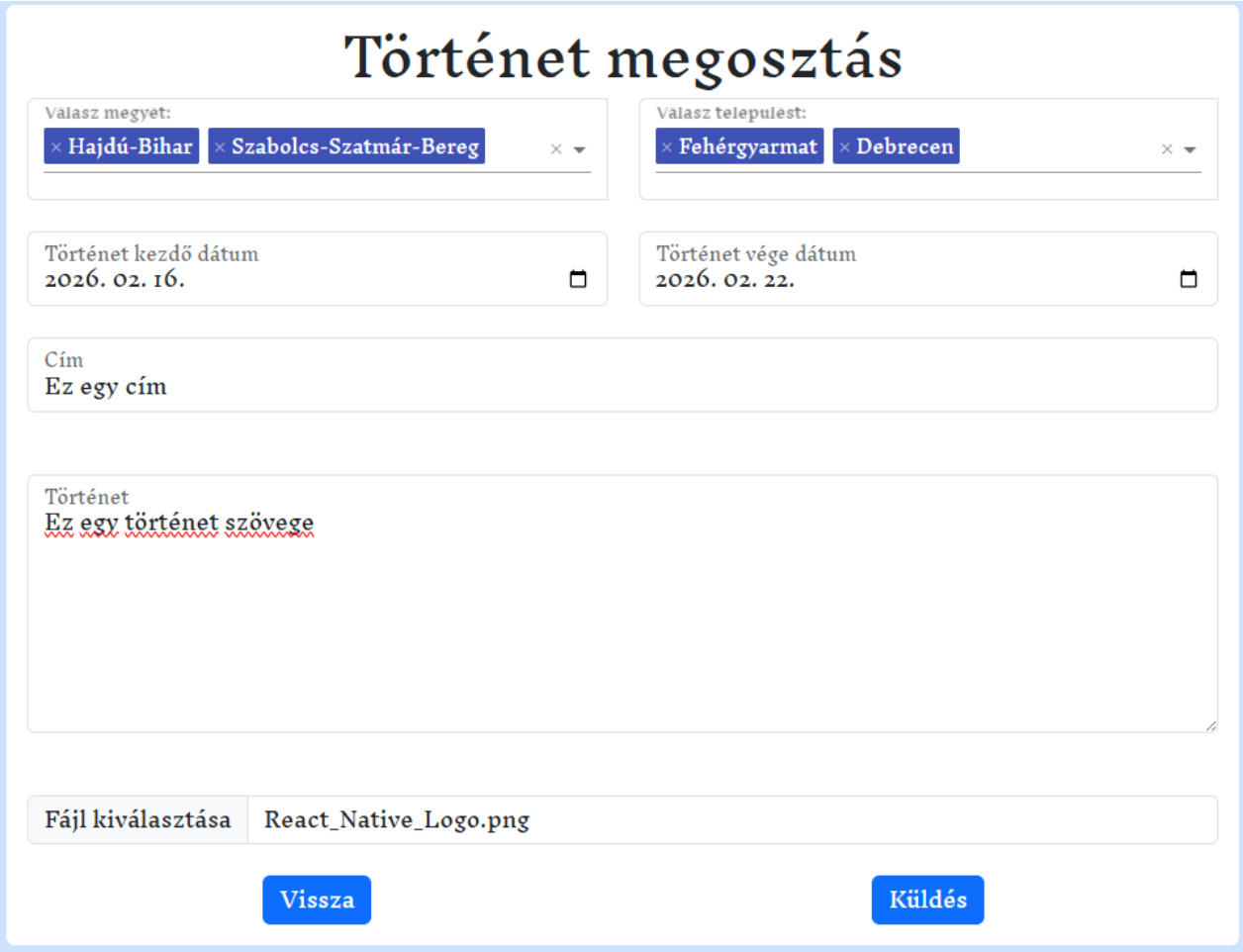
Az oldalon értékelési rész is található, ahol a bejelentkezett felhasználók “tetszik” vagy “nem tetszik” gombot nyomhatnak. Itt látható az is, hogy összesen hányan értékelték már a történetet. Itt hozzászólásokat is írhatnak, amelyek időrendi sorrendben jelennek meg.

Megvalósítás részletei

Amikor a felhasználó megnyitja ezt az oldalt, a frontend elkéri a backend-től a történet összes adatát, amit a backend visszaküld. A frontend ezeket az adatokat megjeleníti.

Amikor valaki értékel, a frontend elküldi az értékelést a backend-nek, amely elmentik az adatbázisba. Ezután a frontend automatikusan frissíti a megjelenített értékelések számát, anélkül hogy újratöltené az egész oldalt. Ugyanígy működik a hozzászólás funkció is.

1.6 Új történet írása (Tortenet-megosztas komponens)



4. ábra

Működés

Ez az oldal csak bejelentkezett felhasználók számára érhető el. Itt lehet új történeteket felvinni a rendszerbe. Az oldal egy űrlapot tartalmaz, ahol a felhasználó kitöltheti a történet minden részletét.

Az űrlap mezői a következők: megye választása: egy legördülő lista, ahol ki kell választani, hogy melyik megyében található a település; település választása: miután kiválasztották a megyét, megjelennek az ahhoz tartozó települések, és ki kell választani legalább az egyiket, de többet is lehet, ahogy megyét is; történet kezdő dátuma: hogy mikor kezdődött a történet; történet vége dátuma: a történet befejezésének dátuma; cím: a történet rövid, találó címe/neve; történet szövege: a tényleges történet teljes szövege; kép feltöltése (opcionális): ha van kép, azt itt lehet feltölteni.
Ha a felhasználó minden mezőt kitöltött, a “Küldés” gombra kattintva elküldheti a történetet.

Ha valamelyik kötelező mező hiányzik vagy hibás, hibaüzenet jelenik meg, ami megmutatja, mit kell javítani.

Megvalósítás részletei

A megye és település választás egy különleges funkcióval rendelkezik: amikor a felhasználó kiválaszt egy megyét, akkor a frontend azonnal lekéri a backend-től az ahhoz a megyéhez tartozó összes települést. Ez dinamikusan történik, tehát az oldal nem töltődik újra, csak a település lista frissül.
Amikor a felhasználó megnyomja a "Küldés" gombot, a frontend először helyben ellenőrzi az űrlap mezőit. Ellenőrzi, hogy: van-e kiválasztva megye és település, a cím ezt nem üres-e, a kezdő dátum nem lehet későbbi, mint a vég dátum és nem lehet a jövőben.
Ha bármelyik ellenőrzés hibát jelez, akkor a megfelelő mező mellett piros színnel megjelenik a hibaüzenet. Például: "Kérlek válassz települést!". Ezután a frontend automatikusan átirányítja a felhasználót az újonnan létrehozott történet oldalára.

1.7 Saját történet szerkesztése (Tortenet-szerkesztes komponens)

Működés

Ha egy felhasználó szerkeszteni szeretné a saját, korábban feltöltött történetét, ezen az oldalon teheti meg. Az oldal hasonlít a "Új történet írása" oldalhoz, de itt az űrlap már ki van töltve a történet jelenlegi adataival.
Amikor elkészült a módosításokkal, a "Mentés" gombra kattintva mentheti el a változtatásokat. Csak az a felhasználó érheti el ezt az oldalt, aki eredetileg létrehozta a történetet.

Megvalósítás részletei

Amikor a felhasználó megnyitja a szerkesztő oldalt, a frontend először ellenőrzi, hogy valóban bejelentkezett felhasználó a történet szerzője.

Ha a felhasználó jogosult a szerkesztésre, a frontend lekéri a történet jelenlegi adatait a backend-től, és kitölti velük az űrlap mezőit.

A mentés során ugyanazok az ellenőrzések futnak le, mint új történet létrehozásakor. Ha minden rendben van, a frontend elküldi a módosított adatokat a backend-nek. A backend frissíti az adatbázisban a történetet. A frontend ezután visszairányítja a felhasználót a frissített történet oldalára.

Van egy "Törlés" gomb is ezen az oldalon, amellyel a felhasználó véglegesen törölheti a történetét.

1.8 Összes történet böngészése (Osszes-tortenet komponens)

Működés

Ez az oldal megjeleníti az összes feltöltött történetet, függetlenül attól, hogy melyik településhez tartoznak.

Az oldal tetején található egy keresési funkció, ahol a felhasználó szűrheti a történeteket megye és település szerint. Először kiválaszt egy megyét, majd az ahhoz tartozó települések közül választhat. Amikor megnyomja a “Keresés” gombot, a lista szűkül, és csak az adott településhez tartozó történetek jelennek meg.
Van egy “Szűrés törlése” gomb is, amellyel újra megjelenítheti az összes történetet.

Megvalósítás részletei

Amikor a felhasználó megnyitja ezt az oldalt, a frontend lekéri a backend-től az összes történetet. Ezek egy nagy listában jelennek meg, hasonlóan a település oldalakon látható kártyákhoz.
A keresés funkció különlegessége, hogy nem kell újratölteni az oldalt. Amikor a felhasználó megnyomja a “Keresés” gombot, a frontend helybenszűri a listát. Ez azt jelenti, hogy az összes történet már le van töltve, és a frontend csak eldönti, melyiket jelenítse meg.

1.9 Fiók szerkesztése (Fiok-szerkesztes komponens)

Működés

Ezen az oldalon a bejelentkezett felhasználó módosíthatja a saját fiókadatait. Minden mező ki van töltve a felhasználó jelenlegi adataival (kivéve a jelszót, ami biztonsági okokból nem jelenik meg). A felhasználó bármelyik mezőt módosíthatja, majd a “Mentés” gombra kattintva mentheti el a változtatásokat.

Az oldalon van egy “Fiók törlése” gomb is, amellyel a felhasználó véglegesen törölheti a fiókját.
Ugyanezen az oldalon található a történet szerkesztése lehetőség is, itt kell kiválasztani melyik történetet szeretnénk szerkeszteni.

Megvalósítás részletei

Amikor a felhasználó megnyitja ezt az oldalt, a frontend lekéri a backend-től a bejelentkezett felhasználó adatait, és kitölti velük az űrlap mezőit.
A mentés során a frontend ellenőrzi: az új felhasználónév még nincs-e foglalva (kivéve ha nem változtatta meg), az email cím valódi email formátumban van-e, ha új jelszót adott meg, eléggé biztonságos-e.

Ha minden rendben van, a frontend elküldi a módosított adatokat a backend-nek. A backend frissíti az adatbázisban a felhasználó adatait. Ha a felhasználónév változott, akkor minden helyen módosul, ahol szerepel a felhasználó neve.

A fiók törlése: ha a felhasználó megerősíti a törlést, akkor a backend törli: a felhasználó fiókját, összes általa írt történetet, összes hozzászólását összes értékelését. Ezután a felhasználó automatikusan ki lesz jelentkeztetve.

1.10 Navigációs sáv (Navbar komponens)

Működés

A navigációs sáv az oldal tetején található vízszintes menü, amely minden oldalon ugyanúgy jelenik meg. Ez biztosítja, hogy a felhasználók bármikor, bárhonnan gyorsan el tudjanak navigálni a főbb funkciókhoz.
A navigációs sáv tartalma függ attól, hogy a felhasználó be van-e jelentkezve vagy sem:

Be nem jejelentkezett felhasználóknak: logo (Főoldal link), Összes történet, Bejelentkezés/Regisztráció.
Bejelentkezett felhasználóknak: Ugyanaz, mint kijelentkezve, azzal a különbséggel, hogy a "Bejelentkezés/Regisztráció" gomb helyén a felhasználó neve lesz látható, ami egy legördülő menüként funkciónál, és lehetőséget ad fiók és történet szerkesztésre, valamint kijelentkezésre.

Megvalósítás részletei

A navigációs sáv folyamatosan figyeli, hogy van-e bejelentkezett felhasználó. Ezt az információt a frontend a böngésző memóriájában tárol.

A kijelentkezés gomb megnyomásakor a frontend törli a bejelentkezett felhasználó adatait a böngésző memóriájából, majd visszairányítja a felhasználót a bejelentkezési oldalra. Ezután már a nem bejelentkezett állapotnak megfelelő navigációs sáv jelenik meg.

1.11 Lábléc (Footer komponens)

Működés

A lábléc az oldal alján található, minden oldalon megjelenő rész. Itt szerepelhetnek: a készítő csapat neve, elérhetőségek jogi információkra mutató linkek (pl. Adatvédelmi szabályzat, Felhasználási feltételek).

Megvalósítás részletei

A lábléc egy egyszerű, statikus komponens, ami minden oldalon ugyanúgy jelenik meg. Nem tartalmaz bonyolult logikát, csak megjelenít néhány szöveget és linket.
A láblécet úgy terveztük, hogy informatív legyen, de ne vonja el a figyelmet a fő tartalomról.

2. A BACKEND MŰKÖDÉSE

2.1 Mi a backend és miért fontos?

A backend az a rész, amit a felhasználók nem látnak, de nélküle az oldal nem működne. Ez kezeli az adatok tárolását, feldolgozását és biztonságát. Ha a frontend kér valamit, például egy történet megjelenítését, a backend előkeresi az adatokat az adatbázisból és visszaküldi.
A backend Node.js környezetben íródott, Express keretrendszert használva. Ez egy olyan szerver, amely folyamatosan fut egy számítógépen, és várja, hogy a frontend kéréseket küldjön neki.

2.2 Az adatbázis felépítése

	Tábla
☐	fiokok
☐	hozzaszolasok
☐	likes
☐	telepulesek
☐	tortenet
☐	tortenetek
☐	tortenet_telepules

5. ábra

Az adatbázis MySQL típusú, ami egy táblázatokból álló rendszer. A HistóriaWeb négy fő táblát használ:

Fiokok tábla

Ez tárolja a felhasználói fiókokat. Minden felhasználónak van egy sora ebben a táblában, amely tartalmazza a felhasználó adatait, például az egyedi azonosítóját, nevét, e-mail címét.

Tortenetek tábla

Itt tárolódnak a történetek. Minden történet egy sor, amely tartalmazza például a történet címét, magát a történetet, dátumokat.

Telepulesek tábla

Ez tartalmazza Magyarország összes településének adatait: az egyedi azonosítót, a település nevét, a megye nevét, a szélesség és hosszúság koordinátákat (hogy hova kell jelölni a térképen).

Tortenet_telepules tábla

Ez egy összekötő tábla, amely összekapcsolja a történeteket a településekkel. Egy történet több településhez is tartozhat, és egy településhez több történet is tartozhat. Ez a tábla tárolja ezeket a kapcsolatokat.

2.3 API végpontok (Endpoints)

Az API végpontok olyan címek, amelyekre a frontend kéréseket tud küldeni. Minden végpont egy konkrét feladatot lát el.

Főoldal térképadatai

Végpont: GET /api/terkep

Mit csinál: Visszaadja az összes települést, amelyhez van már feltöltött történet, a hozzájuk tartozó koordinátákkal.

Hogyan működik:

1. A frontend elküldi a kérést
2. A backend lekérdezi az adatbázisból azokat a településeket, amelyekhez van történet
3. Visszaküldi a települések listáját a nevükkel, megyéjükkel és koordinátáikkal
4. A frontend ezeket megjeleníti a térképen

Történetek lekérése településenként

Végpont: GET /api/tortenet/:telepules

Mit csinál: Visszaadja az összes történetet, amely az adott településhez tartozik.

Hogyan működik:
1. A frontend elküldi a település nevét (például "Budapest")
2. A backend megkeresi az adatbázisban az összes olyan történetet, amely ehhez a településhez kapcsolódik
3. Visszaküldi a történetek listáját címmel, szöveggel, dátummal és képpel
4. A frontend kártyákban megjeleníti őket

Egyedi történet lekérése

Végpont: GET /api/tortenet/telepules/:id

Mit csinál: Visszaadja egy konkrét történet összes adatát.

Hogyan működik:
1. A frontend elküldi a történet egyedi azonosítóját
2. A backend megkeresi ezt a történetet az adatbázisban
3. Lekéri hozzá az összes kapcsolódó adatot (szerző neve, hozzászólások, értékelések)
4. Visszaküldi az összes információt
5. A frontend szépen formázva megjeleníti

Bejelentkezés

Végpont: GET /api/fiokok?felhasznalonev=…&jelszo=…

Mit csinál: Ellenőrzi a felhasználónevet és jelszót, és ha helyesek, visszaadja a felhasználó adatait.

Hogyan működik:
1. A frontend elküldi a beírt felhasználónevet és jelszót

2. A backend megnézi, van-e ilyen felhasználó az adatbázisban
3. Ha van, összehasonlítja a beírt jelszót a tárolt jelszóval
4. Ha egyeznek, visszaküldi a felhasználó adatait
5. Ha nem egyeznek, hibaüzenetet küld
6. A frontend ennek megfelelően vagy belépteti a felhasználót, vagy hibaüzenetet jelenít meg.

Regisztráció

Végpont: POST /api/fiokok

Mit csinál: Létrehoz egy új felhasználói fiókot.

Hogyan működik:
1. A frontend elküldi az új felhasználó adatait (felhasználónév, email, jelszó)
2. A backend ellenőrzi, hogy a felhasználónév még nincs-e foglalva
3. Ha szabad, létrehoz egy új sort a fiokok táblában
4. A jelszót titkosított formában tárolja (biztonsági okokból)
5. Visszaküldi az új felhasználó adatait 6. A frontend automatikusan belépteti az új felhasználót

Új történet létrehozása

Végpont: POST /api/tortenetek/:id

Mit csinál: Létrehoz egy új történetet az adatbázisban.

Hogyan működik:
1. A frontend elküldi a történet összes adatát (cím, szöveg, dátum, kép, települések)
2. A backend létrehoz egy új sort a tortenetek táblában
3. Beállítja a szerző azonosítóját a bejelentkezett felhasználó azonosítójára
4. Minden megadott településhez létrehoz egy kapcsolatot a tortenet_telepules táblában
5. Visszaküldi az újonnan létrehozott történet adatait
6. A frontend átirányítja a felhasználót az új történet oldalára

Történet módosítása

Végpont: PUT /api/tortenetek/:id

Mit csinál: Frissíti egy meglévő történet adatait.

Hogyan működik:
1. A frontend elküldi a módosított adatokat
2. A backend ellenőrzi, hogy a bejelentkezett felhasználó-e a történet szerzője
3. Ha igen, frissíti a történet adatait az adatbázisban
4. Ha a települések listája változott, törli a régi kapcsolatokat és létrehozza az újakat
5. Visszaküld egy megerősítést
6. A frontend visszairányítja a felhasználót a frissített történethez

Történet törlése

Végpont: DELETE /api/tortenetek/:id

Mit csinál: Törli egy történetet az adatbázisból.

Hogyan működik:
1. A frontend elküldi a törlendő történet azonosítóját
2. A backend ellenőrzi, hogy a bejelentkezett felhasználó-e a szerzője
3. Ha igen, először törli az összes kapcsolatot a tortenet_telepules táblából
4. Ezután törli magát a történetet
5. Visszaküld egy megerősítést
6. A frontend visszairányítja a felhasználót a főoldalra

Megyék lekérése

Végpont: GET /api/megyek

Mit csinál: Visszaadja Magyarország összes megyéjének listáját.

Hogyan működik:
1. A frontend elküldi a kérést
2. A backend lekérdezi az adatbázisból az összes egyedi megyenevet
3. ABC sorrendbe rendezi őket
4. Visszaküldi a listát
5. A frontend megjeleníti őket egy legördülő listában

Települések lekérése megyénként

Végpont: GET /api/telepulesek/:megye

Mit csinál: Visszaadja egy adott megye összes településének listáját.

Hogyan működik:
1. A frontend elküldi a megye nevét
2. A backend lekérdezi az adatbázisból az ehhez a megyéhez tartozó összes települést 3. ABC sorrendbe rendezi őket
4. Visszaküldi a listát
5. A frontend megjeleníti őket egy legördülő listában

Összes történet lekérése

Végpont: GET /api/osszes

Mit csinál: Visszaadja az adatbázisban található összes történetet.

Hogyan működik:
1. A frontend elküldi a kérést
2. A backend lekérdezi az adatbázisból az összes történetet
3. Időrendi sorrendbe rendezi őket (legújabb először)
4. Visszaküldi a teljes listát
5. A frontend kártyákban megjeleníti őket

Jogosultságok ellenőrzése

Bizonyos műveletek csak bejelentkezett felhasználók számára elérhetők. A backend minden ilyen kérésnél ellenőrzi, hogy a felhasználó be van-e jelentkezve. Ha nem, akkor nem ad jogosultságot a felhasználónak az adott cselekedetre.

Továbbá a saját történetek szerkesztése és törlése csak a történet szerzője számára engedélyezett. A backend minden ilyen kérésnél ellenőrzi, hogy a bejelentkezett felhasználó azonosítója megegyezik-e a történet szerzőjének azonosítójával.

2.5 Adatbázis kapcsolat kezelése

A backend egy állandó kapcsolatot tart fenn a MySQL adatbázissal. Amikor elindul a szerver, először csatlakozik az adatbázishoz. Ha a kapcsolat sikeres, a szerver készen áll a kérések fogadására.

Minden kérés érkezésekor a backend elküldi a megfelelő SQL lekérdezést az adatbázisnak. Az SQL (Structured Query Language) egy olyan nyelv, amelyen az adatbázissal lehet kommunikálni. Például:

SELECT*FROM tortenetek WHERE telepules ='Budapest'

Ez a lekérdezés azt jelenti: “Válaszd ki az összes történetet, ahol a település Budapest”.

Az adatbázis végrehajtja a lekérdezést, majd visszaküldi az eredményt a backend-nek. A backend ezután továbbítja ezt az eredményt a frontend-nek.

3. HASZNÁLT TECHNOLÓGIÁK ÉS ESZKÖZÖK

3.1 Angular keretrendszer

```
const routes: Routes = [
  {path: "", component: FooldalComponent},
  {path: "hitelesito", component: HitelesitoComponent, canActivate:[Kijelentkezett]},
  {path: "tortenetek/:telepules/:id", component: TortenetComponent},
  {path: "tortenet_megosztas", component: TortenetMegosztasComponent,canActivate:[Bejelentkezett]},
  {path: "tortenetek/:telepules", component: TortenetekComponent},
  {path: "fiok_szerkesztes",component:FiokSzerkesztesComponent,canActivate:[Bejelentkezett]},
  {path:"tortenet_szerkesztes/:id",component:TortenetSzerkesztesComponent,canActivate:[Bejelentkezett]},
  {path: "osszes", component: OsszesTortenetComponent}
];
```

6. ábra

Mi az Angular?

Az Angular egy modern webalkalmazás-fejlesztő keretrendszer, amelyet a Google fejleszt és tart karban. Legnagyobb előnye az SPA (Single Page Application) technológia, amely gyorsabb, gördülékenyebb felhasználói élményt nyújt azáltal, hogy csak a változó részek frissülnek, nem töltődik újra az egész oldal.

A keretrendszer a TypeScript nyelvre épül, és komponensalapú felépítést használ, amely megkönnyíti a nagyobb alkalmazások átlátható és hatékony fejlesztését. Az Angular számos beépített funkciót kínál, például útvonalkezelést és adatkezelést, így alkalmas összetettebb, skálázható webalkalmazások készítésére.

Miért választottuk az Angulart?

A HistóriaWeb fejlesztésekor több keretrendszert is megvizsgáltunk, de végül az Angular mellett döntöttünk:

Strukturáltság: Erősen strukturált, komponensalapú architektúrát biztosít, amely átláthatóbbá és könnyebben karbantarthatóvá teszi a fejlesztést.

TypeScript: A típusos nyelv használata biztonságosabb programozást tesz lehetővé, valamint segít elkerülni a tipikus JavaScript hibákat.

Teljes ökoszisztéma: Beépítetten tartalmazza a legfontosabb funkciókat, mint az útvonalkezelés, HTTP-kommunikáció, űrlapkezelés és validáció.

Közösségi támogatás: Széles fejlesztői közösséggel rendelkezik, amelynek köszönhetően számos dokumentáció és útmutató érhető el.

Az Angular a projektünkben

Komponensek

Minden vizuális elem egy komponens (FooldalComponent, NavbarComponent, TortenetComponent stb.).
Minden komponens három részből áll:
-TypeScript fájl: Logika, funkciók, adatkezelés
-HTML fájl: Vizuális megjelenés
-CSS fájl: Stílus, formázás

Routing (útvonalkezelés)

Az Angular routing rendszere azt határozza meg, hogy az alkalmazás melyik URL-hez melyik oldal (komponens) tartozik. Ennek segítségével az oldal újratöltése nélkül lehet az alkalmazáson belül navigálni.

Példák az alkalmazásban használt útvonalakra:

/ → Főoldal

/tortenetek/Budapest → Egy adott településhez tartozó történetek megjelenítése

/tortenet_megosztas → Új történet írására szolgáló oldal

Ez a megoldás gyorsabb és felhasználóbarátabb működést biztosít.

Service-ek (szolgáltatások)

A service-ek olyan Angular elemek, amelyek az üzleti logikát és az adatkezelést végzik. Segítségükkel az adatok kezelése elkülönül a megjelenítéstől.

A projektben használt főbb service-ek:

TortenetekService:
A történetekkel kapcsolatos backend kommunikációért felel, például történetek lekéréséért egy adott település alapján.

```
getTortenetekByTelepules(telepules: string) {
  // Backend kérés
}
```

BejelentkezesService: Bejelentkezés kezelése és felhasználói állapot tárolása.

TelepulesekService: Megyék és települések lekérése.

Data Binding (adatkötés)

Az adatbinding biztosítja az automatikus kapcsolatot az adatok és a megjelenítés között. Ha a felhasználó módosít egy mezőt, az adat azonnal frissül a programban is.

Példa:

```
<input [(ngModel)]="fiok.felhasznalonev">
```

Ez azt jelenti, hogy a beviteli mező és a felhasznalonev változó mindig szinkronban van.

Direktívák

A direktívák segítségével lehet az HTML elemek viselkedését vezérelni.

*ngFor:
Lista elemeinek megjelenítésére szolgál.

```
<div *ngFor="let tortenet of tortenetek">
  <h3>{{ tortenet.cim }}</h3>
</div>
```

Ez minden történetet külön megjelenít a listából.

*ngIf:
Feltételes megjelenítést tesz lehetővé.

```
<button *ngIf="bejelentkezett">Kijelentkezés</button>
```

A gomb csak akkor látszik, ha a felhasználó be van jelentkezve.

Angular CLI

Az Angular CLI egy parancssori eszköz, amely megkönnyíti a projekt létrehozását és kezelését.

Leggyakrabban használt parancsok:

Új projekt létrehozása:
ng new projekt-nev

Új komponens generálása:
ng generate component komponens-nev

Fejlesztői szerver indítása:
ng serve

Alkalmazás fordítása (build):
ng build

3.2 TypeScript programozási nyelv

```
@Component({
  selector: 'app-footer',
  standalone: false,
  templateUrl: './footer.component.html',
  styleUrl: './footer.component.css'
})
export class FooterComponent {

}
```

7. ábra

Mi a TypeScript?

A TypeScript a JavaScript fejlettebb változata, amelyet a Microsoft fejlesztett ki 2012-ben. Célja, hogy biztonságosabbá és átláthatóbbá tegye a webes alkalmazások fejlesztését. Legfontosabb különbsége a JavaScripthez képest a típusbiztonság, amely előre meghatározza, hogy egy változó milyen típusú adatot tárolhat.

Előnyök

Típusbiztonság

A TypeScript megakadályozza, hogy egy változó nem megfelelő típusú értéket kapjon, így sok hiba már a fejlesztés során kiszűrhető.

Példa:

let szam: number = 5;
szam = "öt"; // Hiba! Nem lehet szöveg egy number változóba

Ez segít elkerülni a futás közbeni hibákat.

Automatikus kiegészítés

A fejlesztői környezet (IDE) ismeri a változók és objektumok típusait, ezért automatikusan felajánlja az elérhető tulajdonságokat és metódusokat. Ez gyorsabb és pontosabb munkavégzést tesz lehetővé.

Ez biztosítja, hogy minden történet azonos felépítésű legyen.

TypeScript a projektünkben

Ez segít az adatok egységes és átlátható kezelésében.

Típusbiztonság a service-ekben

A service-ekben a TypeScript biztosítja, hogy a backendből érkező adatok megfelelő típusúak legyenek.

```
getTortenetekByTelepules(telepules:string):Observable<any>{
return this.http.get(`${this.api}/tortenet/${telepules}`);
}
```

Ez azt jelenti, hogy a metódus egy történeteket tartalmazó listát ad vissza, ami csökkenti a hibák lehetőségét.

3.3 Node.js backend környezet

Mi a Node.js?

A Node.js egy JavaScript futtatókörnyezet, amely lehetővé teszi JavaScript kód futtatását böngészőn kívül, szerveroldalon. Ennek segítségével a webalkalmazások backend része is JavaScript nyelven valósítható meg, amely egyszerűbbé teszi a fejlesztést.

Előnyei

JavaScript mindenhol:
Ugyanaz a programozási nyelv használható a frontend és a backend fejlesztése során is.

Gyors működés:
Eseményvezérelt és aszinkron felépítésének köszönhetően hatékonyan kezeli az egyidejű kéréseket.

NPM (Node Package Manager):

Az NPM a Node.js csomagkezelője, amely külső könyvtárak telepítésére és kezelésére szolgál. Segítségével kész, mások által írt megoldások használhatók fel, így nem kell mindent a nulláról megvalósítani. A használt csomagokat a package.json fájl tartalmazza, ami megkönnyíti a projekt újratelepítését.

Aktív közösség:
Széles körű és folyamatosan frissülő dokumentáció és közösségi támogatás áll rendelkezésre.

Node.js a projektünkben

Szerver létrehozása

A backend alapját egy Express keretrendszerrel létrehozott szerver képezi, amely a kliens kéréseit kezeli.

```
const express = require('express');
const app = express();
const port = 3000;

app.listen(port, () => {
  console.log(`Szerver fut a ${port} porton`);
});
```

Ez a kód elindítja a szervert a megadott porton.

API végpontok

Az alkalmazás a frontend és a backend közötti kommunikációt REST API végpontokon keresztül valósítja meg. Ezek az útvonalak határozzák meg, hogy a kliens milyen módon kérhet le vagy küldhet adatokat a szervernek.

```
app.get('/api/tortenetek', (req, res) => {
  // Történetek lekérése
});
```

Ez a végpont a történetek lekérésére szolgál.

```
app.post('/api/tortenet', (req, res) => {
  // Új történet mentése
});
```

Ez a végpont új történet mentését teszi lehetővé. Ezek az útvonalak biztosítják az adatok lekérését és mentését az alkalmazásban.

Middleware-ek

A middleware-ek a beérkező kérések feldolgozásában részt vevő köztes elemek, amelyek a kérés és a válasz között futnak le.

body-parser:
A kliens által küldött JSON adatok feldolgozására szolgál.

cors:
Lehetővé teszi, hogy a frontend és a backend különböző domainekről is kommunikálhasson egymással.

express.static:
Statikus fájlok, például képek vagy egyéb erőforrások kiszolgálását végzi.

3.4 Express.js keretrendszer

Mi az Express.js?

Az Express.js egy könnyen használható Node.js webalkalmazás-keretrendszer, amely leegyszerűsíti a szerveroldali alkalmazások fejlesztését, különösen az útvonalkezelést és a kérések feldolgozását.

Miért használtuk?

Egyszerű és átlátható routing

Middleware-ek hatékony használata

Gyors fejlesztési lehetőség

Jól dokumentált és széles körben használt keretrendszer

Express.js a projektünkben

Routing

A routing külön fájlokba szervezve valósul meg, ami átláthatóbb kódszerkezetet eredményez.

```
const tortenetekRouter = require('./routes/tortenetek');
app.use('/api/tortenetek', tortenetekRouter);
```

3.5 MySQL adatbázis

Mi a MySQL?

A MySQL egy nyílt forráskódú, relációs adatbázis-kezelő rendszer, amely strukturált adatok tárolására és kezelésére szolgál. Az adatokat táblákban rendezi, ahol az egyes rekordok egymással kapcsolatba hozhatók. A MySQL az egyik legelterjedtebb adatbázis-megoldás webes alkalmazások esetén.

Miért MySQL?

Ingyenes és nyílt forráskódú: Szabadon használható és módosítható.

Gyors és megbízható működés: Nagy mennyiségű adat hatékony kezelésére is alkalmas.

Széles körű támogatás: Számos programozási nyelv és keretrendszer támogatja.

XAMPP környezettel kompatibilis: Egyszerűen használható fejlesztési környezetben.

Adatbázis struktúra

Az adatbázis több, egymással kapcsolatban álló táblából épül fel, amelyek különböző típusú adatokat tárolnak. Ez a felépítés biztosítja az adatok áttekinthetőségét és hatékony kezelését.

A két legfőbb tábla

fiok tábla:
A felhasználók adatait tárolja.

id (int, primary key)

felhasznalonev (varchar)

email (varchar)

jelszo (varchar)

tortenet tábla:
A felhasználók által létrehozott történetek adatait tartalmazza.

id (int)

fiok_id (int, foreign key)

cim (varchar)

tortenet (varchar)

keletkezes_datum (datetime)

tortenet_datum_kezdet (datetime)

tortenet_datum_vege (datetime)

kep_url (varchar)

MySQL kapcsolat Node.js-ben

A Node.js alapú backend alkalmazás egy dedikált adatbázis-kapcsolaton keresztül kommunikál a MySQL adatbázissal. Ez teszi lehetővé az adatok lekérését és mentését.

```
const mysql = require('mysql2');

const db = mysql.createConnection({
host: "localhost",
user: "root",
password: "",
database: "historiaweb"

});



db.connect(err => {
if (err) throw err;
console.log("MySQL kapcsolódva!");
});
```

Ez a kapcsolat biztosítja az alkalmazás és az adatbázis közötti folyamatos adatcserét.

3.6 Bootstrap CSS keretrendszer

Mi a Bootstrap?

A Bootstrap egy előre elkészített CSS keretrendszer, amely előre definiált stílusokat és vizuális komponenseket biztosít a weboldalakhoz. Segítségével gyorsan lehet szép, egységes és reszponzív (különböző képernyőméretekhez igazodó) felületeket készíteni, anélkül, hogy minden egyes stílust nulláról kellene megírni.

Előnyei

Reszponzív design: Az oldal automatikusan alkalmazkodik mobil, tablet vagy asztali képernyőhöz.

Grid rendszer: Egyszerűen lehet elrendezéseket létrehozni oszlopokkal és sorokkal.

Komponensek: Kész gombok, űrlapok, kártyák, modálok használhatók.

Gyors fejlesztés: Kevesebb egyedi CSS-t kell írni, így a fejlesztés sokkal gyorsabb.

Bootstrap a projektünkben

Grid rendszer

```
<div class="container">
  <div class="row">
    <div class="col-md-8">Fő tartalom</div>
    <div class="col-md-4">Oldalsáv</div>
  </div>
</div>
```

container: A teljes tartalom kerete.

row: Egy sor a rácsban.

col-md-8 / col-md-4: Az oszlopok szélességét jelöli, 8 és 4 arányban.

Használt komponensek

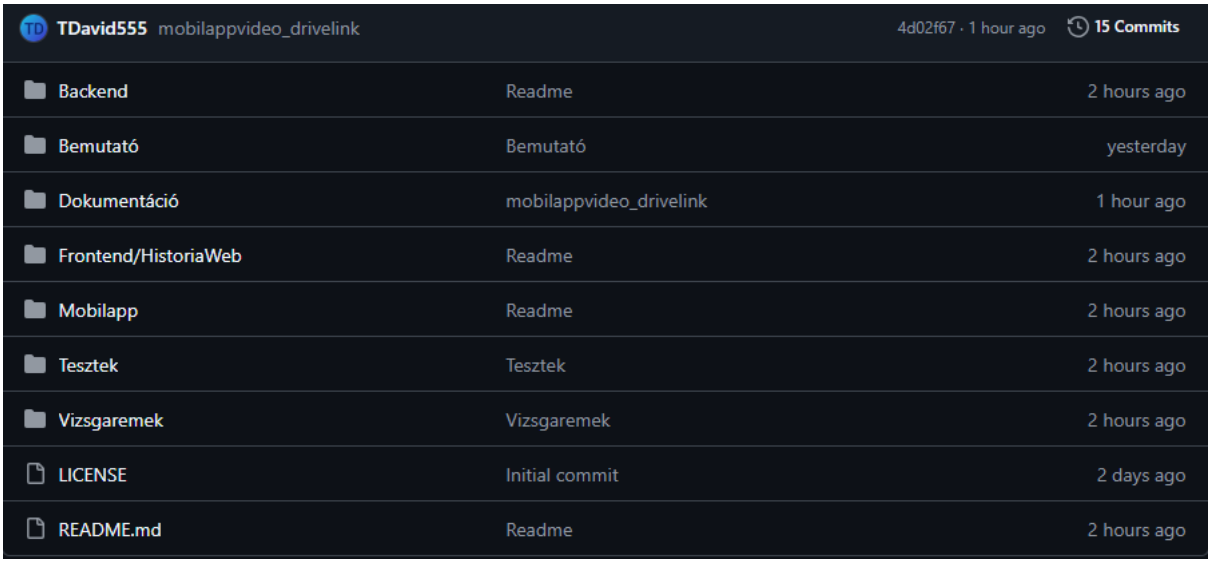
Navbar: Navigációs sáv az oldal tetején.

Card: Történetek megjelenítése egy rendezett dobozban.

Modal: Felugró ablak bejelentkezéshez vagy regisztrációhoz.

Form: Űrlapok gyors létrehozása egységes stílusban.

3.7 GitHub – verziókezelés



8. ábra

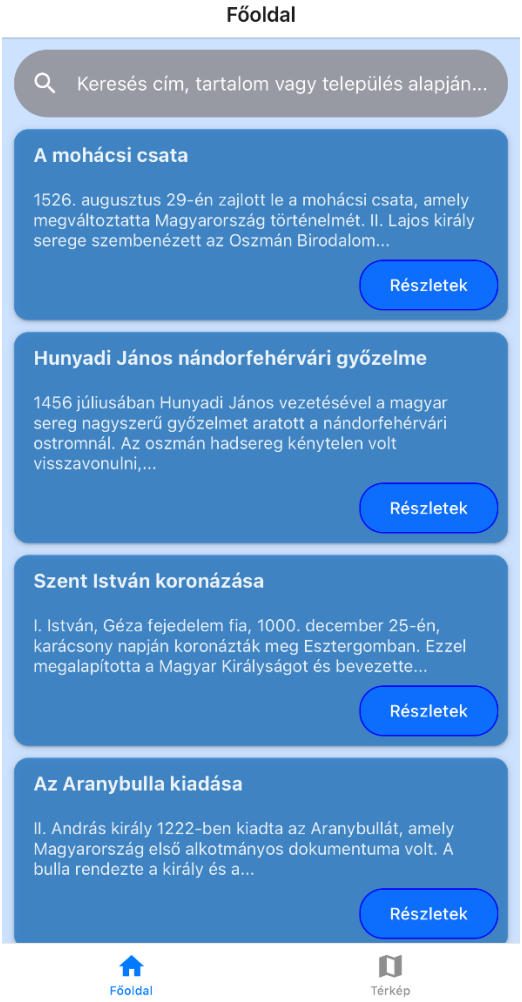
Mi a GitHub?

A GitHub egy online felület, ahol a Git projektek tárolhatók. Itt a csapatok közösen dolgozhatnak a kódon, és a változtatások nyomon követhetők.

Alapvető parancsok

git add . - Fájlok hozzáadása a verziókezeléshez
git commit -m "üzenet" - A változtatások rögzítése
git push origin main - Feltöltés a GitHub-ra
git pull origin main - Frissítés a GitHub-ról
git branch feature - Új ág (branch) létrehozása

3.8 React Native mobilalkalmazás



9. ábra

Mi a React Native?

A React Native egy nyílt forráskódú keretrendszer mobilalkalmazások fejlesztéséhez, amelyet a Facebook (Meta) hozott létre. Lehetővé teszi, hogy JavaScript és a React könyvtár használatával natív mobilalkalmazásokat készítsünk Android és iOS platformokra. A React Native nem webes alkalmazásokat futtat egy böngészőben, hanem valódi natív komponenseket használ, ezért az elkészült alkalmazások teljesítménye közel áll a natív appokéhoz.

Előnyei

Egy kódbázis több platformra:
A legtöbb kód megosztható Android és iOS között, ami jelentősen csökkenti a fejlesztési időt és költséget.

Elég a JavaScript és React ismerete:
Nem szükséges külön natív nyelveket tanulni, így webfejlesztők számára gyors az átállás mobilfejlesztésre.

Hot reload / Fast refresh:
A kódban végrehajtott módosítások azonnal megjelennek a futó alkalmazásban, ami felgyorsítja a fejlesztést és a hibakeresést.

Natív kinézet és működés:
Az alkalmazások valódi natív UI-elemeket használnak, így a felhasználói élmény megfelel az adott platform elvárásainak.

Nagy közösség és sok dokumentáció:
Rengeteg oktatóanyag, könyvtár és kész megoldás érhető el, ami megkönnyíti a tanulást és a problémák megoldását.

A HistóriaWeb mobilalkalmazás

A projekthez készítettünk egy React Native alapú mobilalkalmazást is, amelyet az Expo keretrendszer segítségével fejlesztettünk. Az alkalmazás Android és iOS platformon egyaránt futtatható, egyetlen közös kódbázisból. Az app a webes rendszerrel azonos backend API-t használja, így a felhasználók a mobilon is hozzáférhetnek a HistóriaWeb tartalmaihoz.

Az alkalmazás felépítése és képernyői

Az app Expo Router alapú fájl-alapú útvonalkezelést használ. Az alkalmazás indításkor automatikusan a Főoldal fülre navigál. Két fő fül érhető el az alsó navigációs sávon keresztül: a Főoldal és a Térkép.

Főoldal (fooldal.tsx): A főoldalon az összes történet listája jelenik meg kártyás elrendezésben, kékes átlátszó háttérrel. Minden kártyán látható a történet címe és egy rövid előnézeti szöveg (az első 20 szó). A keresősáv segítségével a felhasználók cím, tartalom vagy település szerint szűrhetnek. A Részletek gombra koppintva megnyílik az adott történet teljes nézete. A képernyő betöltésekor az alkalmazás automatikusan frissíti az összes történetet a szerverről, és lehetővé teszi az álló és fekvő tájolást egyaránt.

Térkép (terkep.tsx): A térképes nézet Magyarország SVG-alapú, interaktív megyetérképét jeleníti meg, amely nagyítható és mozgatható (pinch-to-zoom, illetve húzás gesztusokkal). A térkép automatikusan fekvő tájolásra vált, amikor a felhasználó erre a fülre lép. Az egyes történetek helyszínei piros helyjelölő ikonokkal vannak feltüntetve a megfelelő koordinátákon. Egy-egy helyszínre koppintva az alkalmazás az adott településhez tartozó történetek listájára navigál.

Történetek település szerint (tortenetek.tsx): Ha a felhasználó a térképen kiválaszt egy települést, ez a képernyő jelenik meg, amely az adott helyszínhez tartozó összes történetet listázza. A főoldalhoz hasonlóan itt is elérhető a keresési funkció (cím vagy tartalom alapján), és minden kártyáról meg lehet nyitni az adott történet részletes nézetét.

Részletes nézet (details.tsx): Az egyes történetek teljes szövege, szerzője, keletkezési dátuma és a történés időszaka jelenik meg itt. A Vissza gombbal a felhasználó visszatérhet az előző listanézethez.

Technikai megvalósítás

Az adatok kezelése service rétegen keresztül történik: a tortenet.service.ts a történeteket, a pont.service.ts a térképpontokat kezeli, mindkettő a React useSyncExternalStore hook segítségével biztosít reaktív, valós idejű állapotkezelést. Az app.state.ts egy egyszerű globális állapotot valósít meg a kiválasztott település és történet tárolásához. A hálózati kommunikáció az api.service.ts-en keresztül zajlik, amely egy konfigurálható szerver URL-t használ; ha a szerver nem érhető el, az alkalmazás automatikusan helyi (localhost) kapcsolatra vált. Az allas.service.ts az Expo Screen Orientation könyvtáron keresztül kezeli a képernyő tájolásának automatikus beállítását az egyes nézetekhez. A felhasználói felület a React Native Paper komponenskönyvtárra épül (kártyák, keresősáv, gombok), a stílusok a styles.ts fájlban vannak összegyűjtve, és a téma automatikusan alkalmazkodik a sötét és világos módhoz egyaránt.

3.9 Tesztelés – Jasmine és Jest

A szoftverfejlesztés egyik kulcslépése a tesztelés, amely biztosítja, hogy a program helyesen működjön és a későbbi módosítások ne törjék el a már elkészült funkciókat. A HistóriaWeb projektben frontend és backend oldalon is tesztelést alkalmazunk, két különböző keretrendszerrel: Jasmine és Jest.

Frontend tesztelés – Jasmine

Mi a Jasmine?

A Jasmine egy JavaScript tesztelési keretrendszer, amely különösen az Angular projektekhez használatos. Lehetővé teszi a komponensek és szolgáltatások viselkedésének ellenőrzését, biztosítva, hogy a felhasználói felület és a logika megfelelően működjön.

Miért fontos?

Ellenőrzi, hogy a komponensek a várt módon jelennek-e meg.

Biztosítja, hogy a szolgáltatások (pl. adatkezelés, bejelentkezés) helyesen működjenek.

Segít a hibák korai felismerésében, még a fejlesztés közben.

Példa teszt: BejelentkezesService

```
describe('BejelentkezesService', () => {
  let service: BejelentkezesService;
  beforeEach(() => {
    TestBed.configureTestingModule({});
```

```
    service = TestBed.inject(BejelentkezesService); // Service példány lekérése
  });
  it('should be created', () => {
    expect(service).toBeTruthy(); // Ellenőrzés: a service sikeresen létrejött
  });
});
```

Magyarázat:

describe: Egy tesztcsoportot hoz létre, ami logikailag összefogja az adott szolgáltatás tesztjeit.

it: Egy konkrét teszt eset, ami leírja, mit várunk a kódtól.

expect: Ellenőrzi, hogy a tényleges kimenet megegyezik-e a várt értékkel.

Ez a teszt biztosítja, hogy a BejelentkezesService sikeresen példányosítható az Angular dependency injection rendszerén keresztül, és a service készen áll a használatra.

Backend tesztelés – Jest

Mi a Jest?

A Jest egy JavaScript tesztelési keretrendszer, amit elsősorban Node.js szerverekhez használnak. Lehetővé teszi, hogy a backend API végpontokat és logikát automatikusan teszteljük.

Miért fontos?

Ellenőrzi, hogy a szerver megfelelően válaszol a kérésekre.

Biztosítja, hogy az adatbázis lekérdezések és az adatok feldolgozása helyes legyen.

Segít megelőzni, hogy a kódfrissítések megszakítsák az API működését.

Példa teszt: GET /api/fiokok/:id

```
describe('Kapcsolat tesztek', () => {
  test('GET /api/fiokok/:id megfelelő adatokkal tér vissza', async () => {
    const response = await fetch('http://localhost:3000/api/fiokok/1'); // Kérés az API-hoz
    expect(response.status).toBe(200); // Ellenőrizzük, hogy sikeres volt-e
  });
  test('GET /api/fiokok/:id 404-et ad hibás ID-ra', async () => {
    const response = await fetch('http://localhost:3000/api/fiokok/99'); // Nem létező ID
    expect(response.status).toBe(404); // Ellenőrizzük, hogy 404-et kaptunk-e
  });
});
```

Magyarázat:

fetch(...): Natív HTTP GET kérés a futó szerverhez, külső könyvtár nélkül.

response.status: Ellenőrizzük a szerver által visszaküldött státuszkódot (200 = sikeres lekérdezés, 404 = nem található).

Ez a két teszt biztosítja, hogy a fiók lekérése létező ID-ra sikeresen működjön, nem létező ID-ra pedig a szerver a helyes 404-es hibaüzenettel válaszoljon.

3.10 Visual Studio Code (VS Code)

Mi a VS Code?

A Visual Studio Code (VS Code) egy ingyenes és nyílt forráskódú kódszerkesztő, amelyet a Microsoft fejleszt. Könnyű, gyors és sokoldalú eszköz, amelyet világszerte használnak frontend és backend fejlesztéshez, mobilalkalmazások készítéséhez, valamint különböző webes és szoftverprojektekhez. Több operációs rendszeren is elérhető, például Windows, Linux és macOS platformokon.

A VS Code alapból egy egyszerű kódszerkesztő, de a beépített funkciók és a bővítmények segítségével teljes értékű fejlesztői környezetté (IDE) alakítható.

Miért jó?

Könnyen kezelhető és testreszabható felület:
Letisztult felhasználói felülettel rendelkezik, amely kezdők és haladók számára is jól átlátható. A kinézet, billentyűparancsok és beállítások szabadon módosíthatók.

Rengeteg bővítmény érhető el:
Az Extension Marketplace-en több ezer kiegészítő található különböző programozási nyelvekhez és technológiákhoz, például JavaScript, TypeScript, Python, Angular vagy React támogatáshoz.

Valós idejű hibajelzés és kódsegítség:
A VS Code azonnal jelzi a szintaktikai hibákat, automatikus kódkiegészítést kínál, valamint segít a tiszta és jól strukturált kód írásában.

Beépített verziókezelés:
Natívan támogatja a Git használatát, így a kódváltozások követése, commitolása és kezelése közvetlenül a szerkesztőből elvégezhető.

Gyors és erőforrás-takarékos:
Más teljes IDE-khez képest kevesebb erőforrást használ, mégis sok funkciót kínál.

Hasznos bővítmények a projektben

Angular Language Service:
Angular projektekhez nyújt támogatást, például intelligens kódkiegészítést, hibajelzést és sablonellenőrzést, ami megkönnyíti a fejlesztést.

Prettier:
Automatikusan formázza a kódot egy egységes stílus szerint, így javítja az olvashatóságot és csökkenti a formázási hibákat.

GitLens:
Kibővíti a Git funkciókat, lehetővé teszi a változások részletes követését, megmutatja, hogy egy adott kódsort ki és mikor módosított.

Live Server:
Fejlesztés közben valós időben jeleníti meg a weboldalt a böngészőben, és automatikusan frissíti azt a fájl mentésekor.

3.11 XAMPP fejlesztői környezet

Mi a XAMPP?

A XAMPP egy ingyenes, nyílt forráskódú fejlesztői környezet, amely lehetővé teszi webalkalmazások helyi futtatását és tesztelését. Egyetlen csomagban tartalmazza a legfontosabb szerveroldali technológiákat, így ideális tanuláshoz és fejlesztéshez.

A XAMPP főbb összetevői:

Apache:
Webszerver, amely a webalkalmazások kiszolgálásáért felel. Segítségével a böngészőben megjeleníthető a fejlesztés alatt álló weboldal.

MySQL (MariaDB):
Relációs adatbázis-kezelő rendszer, amely az alkalmazás adatait tárolja és kezeli.

PHP:
Szerveroldali programozási nyelv, amely dinamikus weboldalak és webalkalmazások készítésére szolgál.

phpMyAdmin:
Böngészőből elérhető grafikus felület az adatbázisok kezelésére, amely megkönnyíti a táblák, rekordok és lekérdezések kezelését.

Miért hasznos?

Helyi fejlesztés és tesztelés:
Lehetővé teszi, hogy a projektet internetkapcsolat és éles szerver nélkül, saját gépen futtassuk.

Egyszerű telepítés és használat:
Nem szükséges külön-külön telepíteni és konfigurálni a webszervert, az adatbázist és a PHP-t.

Gyors hibakeresés:
A fejlesztő azonnal látja a változtatások eredményét, ami felgyorsítja a fejlesztési folyamatot.

Kezdők számára ideális:
Tanulási célokra és kisebb projektekhez is jól használható.

3.12 Figma – UI/UX design

Mi a Figma?

A Figma egy modern, böngésző-alapú tervezőeszköz, amelyet elsősorban UI (User Interface) és UX (User Experience) tervezésre használnak. Előnye, hogy telepítés nélkül, online működik, és több felhasználó is dolgozhat benne egyszerre.

Mit tudunk vele csinálni?

Wireframe-ek készítése:
Egyszerű, vázlatos tervek az oldal vagy alkalmazás felépítéséről, amelyek a struktúrát és az elrendezést mutatják meg.

Mockup-ok tervezése:
Részletes, vizuálisan kidolgozott tervek, amelyek közel állnak a végleges megjelenéshez.

Színpaletta és komponens könyvtár létrehozása:
Segít az egységes arculat kialakításában, újrahasználható gombokkal, ikonokkal és elemekkel.

Interaktív prototípusok:
Kattintható tervek készíthetők, amelyek bemutatják az alkalmazás működését.

Csapatmunka támogatása:
Több tervező és fejlesztő valós időben dolgozhat ugyanazon a projekten, kommentekkel és visszajelzésekkel.

3.13 Projektmenedzsment eszközök

Trello

A Trello egy egyszerű, vizuális projektmenedzsment eszköz, amely Kanban-tábla alapú feladatkezelést biztosít. Segítségével átláthatóvá válik a projekt állapota és a feladatok előrehaladása.

A tipikus oszlopok:

To Do:
Tervezett, még el nem kezdett feladatok.

In Progress:
Jelenleg folyamatban lévő munkák.

Testing:
Tesztelés alatt álló funkciók és hibajavítások.

Done:
Elkészült, lezárt feladatok.

Discord

A Discord egy kommunikációs platform, amelyet gyakran használnak fejlesztői csapatok együttműködésre.

Csapatkommunikáció:
Szöveges és hangcsatornák a gyors egyeztetéshez.

Screen sharing:
Képernyőmegosztás bemutatókhoz, hibakereséshez vagy közös fejlesztéshez.

Fájlmegosztás:
Dokumentumok, képek és kisebb kódrészletek gyors megosztása.

3.14 Külső források és adatok

A projekt megvalósításához több külső forrás és adatbázis is felhasználható, amelyek megkönnyítik a fejlesztést és javítják a felhasználói élményt.

GitHub:
Megyék és települések listája CSV formátumban, amely egyszerűen importálható adatbázisba.

Geoapify:
Települések földrajzi koordinátáinak lekérdezése, amely térképes megjelenítésekhez használható.

Wikimedia Commons:
Magyarország térképe SVG formátumban, amely interaktív, webes megjelenítésre alkalmas.

Unsplash:
Ingyenesen felhasználható, magas minőségű fotók weboldalak és alkalmazások számára.

FontAwesome / Bootstrap Icons:
Széles körű ikonkönyvtár, amely segíti az egységes és jól érthető felhasználói felület kialakítását.

3.15 Összefoglalás – Technológiai stack

Frontend: Angular + TypeScript + Bootstrap + Jasmine
Backend: Node.js + Express.js + MySQL2 + Jest
Mobil: React Native + Expo Router + React Native SVG
Adatbázis: MySQL / MariaDB + phpMyAdmin + XAMPP
Fejlesztői eszközök: Git / GitHub, Trello, Figma, Discord, Visual Studio Code

Miért jó a stack?

Egységes nyelv: JavaScript mindenhol

Modern, naprakész technológiák

Jó dokumentáció és nagy közösség

Skálázható, könnyen bővíthető projekt

Tanulságok

Előnyök: - Gyors fejlesztés az előre elkészített komponensekkel - Jól strukturált kód - Könnyű csapatmunka a verziókezeléssel - TypeScript segített elkerülni a hibákat

Kihívások: - Kezdetben meredek tanulási görbe - Sok eszköz összehangolása - Különböző környezetek kezelése

Ha újrakezdenénk: - Még több időt töltenénk a tervezéssel - Korábban kezdenénk el tesztelni - Több pair programming session-t szerveznénk

BEFEJEZÉS

4. MUNKAMEGOSZTÁS

A projekt sikeres befejezéséhez elengedhetetlen volt a hatékony csapatmunka és a jól megtervezett munkamegosztás. A fejlesztőcsapatunk két főből állt, mindketten hozzájárultunk a projekt minden részéhez, azonban bizonyos területeken specializálódtunk.

Kozma Patrik – Backend

Patrik elsődleges felelőssége a szerveroldali logika megvalósítása volt. Ő készítette el a Node.js backend architektúrát és az Express.js API végpontokat. Munkája kiterjedt a biztonsági mechanizmusok implementálására is, beleértve a felhasználói hitelesítést és jogosultságkezelést.

Fő feladatai:

- REST API végpontok fejlesztése
- Adatbázis lekérdezések optimalizálása
- Biztonsági mechanizmusok beépítése
- Szerveroldali hibajavítás és teljesítményoptimalizálás
- Backend unit tesztek írása Jest keretrendszerrel
- Térképes funkció integrálása és interaktivitás biztosítása

Tivadar Dávid – Frontend

Dávid a felhasználói felület kialakításáért és a felhasználói élmény optimalizálásáért volt felelős. Ő készítette el az Angular komponenseket és a reszponzív dizájnt. Továbbá ő koordinálta a React Native mobilalkalmazás fejlesztését is.

Fő feladatai:

- Angular komponensek fejlesztése
- Reszponzív dizájn implementálása Bootstrap segítségével
- Frontend unit tesztek írása Jasmine keretrendszerrel
- React Native mobilalkalmazás fejlesztése
- Projektmenedzsment és feladatok koordinálása Trello segítségével
- Dokumentáció készítése és frissítése

Közös felelősségek

- UI/UX tervezés Figma-ban
- MySQL adatbázis tervezése és implementálása

- Kód review és code quality ellenőrzés
- Verziókezelés Git és GitHub segítségével
- Integrációs tesztelés és hibajavítás

Fejlesztési módszertan és sprintek

A projekt megvalósítása során agilis módszertant alkalmaztunk, amely rugalmasságot és hatékonyságot biztosított a fejlesztési folyamatban. A munkát sprint-ekre osztottuk, amelyek hossza a feladat komplexitásától függött.

Kéthetes sprintek – Egyszerűbb, jól definiált feladatok esetén:

- Frontend komponensek fejlesztése (navigációs sáv, lábléc, űrlapok)
- Egyszerű API végpontok implementálása (GET kérések, adatlekérések)
- UI/UX finomhangolás és reszponzív dizájn javítások
- Adatbázis tábla módosítások és optimalizálások
- Kisebb bugok javítása és kód refaktorálás

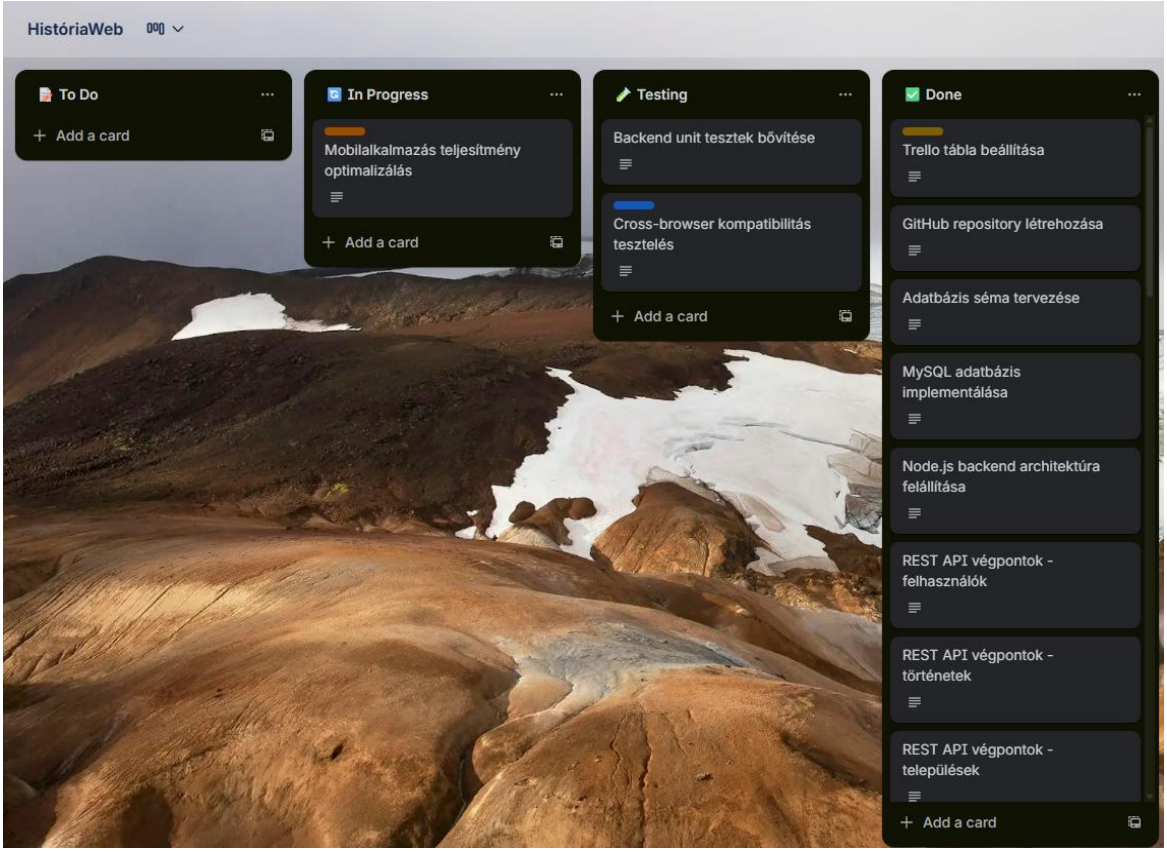
Háromhetes sprintek – Komplexebb, több komponenst érintő feladatok esetén:

- Térképes funkció integrálása (SVG feldolgozás, interaktivitás, adatkötés)
- Hitelesítési rendszer teljes implementálása (frontend és backend együtt)
- React Native mobilalkalmazás fejlesztése és tesztelése
- Képfeltöltési és -kezelési rendszer beépítése
- Adatbázis architektúra tervezése és több tábla közötti kapcsolatok kialakítása

Minden sprint végén **sprint review**-t tartottunk, ahol átbeszéltük az elkészült funkciókat, teszteltük azokat, és meghatároztuk a következő sprint feladatait. A Trello táblán folyamatosan követtük a feladatok státuszát (To Do, In Progress, Testing, Done), ami átláthatóvá tette a projekt előrehaladását.

A **sprint planning** során megbecsültük az egyes feladatok időigényét, és eldöntöttük, hogy kéthetes vagy háromhetes sprintre van szükség. Ha egy sprint során váratlan technikai probléma merült fel, rugalmasan átcsoportosítottuk a feladatokat, és szükség esetén meghosszabbítottuk a sprintet.

Ez a strukturált megközelítés segített fenntartani a fejlesztés lendületét és biztosította, hogy folyamatosan működő funkciókat szállítsunk, amelyeket azonnal tesztelni is tudtunk.



10. ábra

5. KIHÍVÁSOK ÉS MEGOLDÁSOK

A fejlesztési folyamat során számos technikai és szervezési kihívással találkoztunk, amelyekre megoldásokat kellett találnunk.

Térkép használata az oldalon

Az SVG-alapú Magyarország-térképének megjelenítése helyes méretben kihívás volt, mert ezelőtt még nem volt
dolgunk SVG fájllal ilyen formában.

Angular verziók

Eleinte különböző számítógépeket, különböző verzióit használtuk az Angular-nak ami kompatibilitási problémákat eredményezett. Miután rájöttünk, hogy a váratlan problémákat a verzió különbségek okozzák, egységesítettük őket minden általunk használt számítógépen.

6. STATISZTIKAI ADATOK ÉS EREDMÉNYEK

A platform tesztelési fázisában **32 felhasználó** töltötte ki az elégedettségi kérdőívet. A tesztelők különböző korcsoportokból és technikai hozzáértési szintekkel rendelkeztek, hogy átfogó képet kapjunk a platform használhatóságáról.

Demográfiai adatok

Életkor megoszlás:

- 18 alatt: 3 fő (9%)
- 18-25: 14 fő (44%)
- 26-35: 8 fő (25%)
- 36-50: 5 fő (16%)
- 50 felett: 2 fő (6%)

Használt eszközök:

- Számítógép (asztali): 8 fő (25%)
- Laptop: 11 fő (34%)
- Tablet (weboldal): 1 fő (3%)
- Tablet (alkalmazás): 1 fő (3%)
- Mobil (weboldal): 7 fő (22%)
- Mobil (alkalmazás): 4 fő (13%)

Felhasználói élmény értékelése

Használhatóság (Mennyire találta könnyen használhatónak a platformot?):

- Nagyon könnyen használható: 18 fő (56%)
- Könnyen használható: 11 fő (34%)
- Közepesen használható: 2 fő (6%)
- Nehezen használható: 1 fő (3%)
- Nagyon nehezen használható: 0 fő (0%)

Átlagos értékelés: 4.44/5

Dizájn és megjelenés:

- Nagyon elégedett: 15 fő (47%)
- Elégedett: 13 fő (41%)
- Közömbös: 3 fő (9%)
- Elégedetlen: 1 fő (3%)
- Nagyon elégedetlen: 0 fő (0%)

Átlagos értékelés: 4.31/5

Navigáció egyértelműsége:

- Nagyon egyértelmű: 17 fő (53%)
- Egyértelmű: 10 fő (31%)
- Elfogadható: 4 fő (13%)
- Zavaró: 1 fő (3%)
- Nagyon zavaró: 0 fő (0%)

Átlagos értékelés: 4.34/5

Funkciók értékelése

Térképes böngészés:

- Nagyon elégedett: 22 fő (69%)
- Elégedett: 8 fő (25%)
- Közömbös: 2 fő (6%)
- Elégedetlen: 0 fő (0%)
- Nagyon elégedetlen: 0 fő (0%)

Átlagos értékelés: 4.63/5

Keresési/szűrési lehetőségek:

- Nagyon hasznos: 11 fő (34%)
- Hasznos: 16 fő (50%)
- Közepesen hasznos: 4 fő (13%)
- Kevésbé hasznos: 1 fő (3%)
- Egyáltalán nem hasznos: 0 fő (0%)

Átlagos értékelés: 4.16/5

Történet írási és szerkesztési funkció:

- Nagyon elégedett: 12 fő (38%)
- Elégedett: 11 fő (34%)
- Közömbös: 3 fő (9%)
- Elégedetlen: 1 fő (3%)
- Nagyon elégedetlen: 0 fő (0%)
- Nem használtam ezt a funkciót: 5 fő (16%)

Átlagos értékelés (akik használták): 4.22/5

Értékelési rendszer (tetszik/nem tetszik):

- Nagyon értékes: 9 fő (28%)
- Értékes: 14 fő (44%)
- Semleges: 7 fő (22%)
- Kevésbé értékes: 2 fő (6%)
- Felesleges: 0 fő (0%)

Átlagos értékelés: 3.94/5

Teljesítmény és megbízhatóság

Hibák és problémák gyakorisága:

- Egyáltalán nem: 14 fő (44%)
- Ritkán: 13 fő (41%)
- Néha: 4 fő (13%)
- Gyakran: 1 fő (3%)
- Nagyon gyakran: 0 fő (0%)

Ajánlás

Ajánlaná-e másoknak?

- Határozottan igen: 17 fő (53%)
- Valószínűleg igen: 12 fő (38%)
- Talán: 2 fő (6%)
- Valószínűleg nem: 1 fő (3%)
- Határozottan nem: 0 fő (0%)

Jövőbeli funkciók preferenciái (többet is jelölhettek)

1. AI alapú történet összefoglalás: 23 fő (72%)
2. Audio felolvasás: 18 fő (56%)
3. Többnyelvű támogatás: 14 fő (44%)
4. Több kép feltöltése: 13 fő (41%)
5. Gamifikáció (pontok, kitüntetések): 11 fő (34%)
6. Közösségi média megosztás: 10 fő (31%)
7. Helyesírás-ellenőrzés: 8 fő (25%)

Használati gyakoriság

- Naponta: 2 fő (6%)
- Hetente többször: 8 fő (25%)

- Hetente egyszer: 11 fő (34%)
- Havonta néhányszor: 9 fő (28%)
- Ritkábban: 2 fő (6%)

Kiemelkedő pozitívumok

Az értékelések alapján a platform **általános elégedettsége 4.35/5**, ami kiváló értékelés egy új platformnál. A **térképes funkció** kiemelkedően sikeres (4.63/5), amely a platform egyedi értékajánlata.

A tesztelők **91%-a ajánlaná** a platformot másoknak, ami erős bizonyítéka annak, hogy a HistóriaWeb valós felhasználói igényt elégít ki. A leggyakrabban kért jövőbeli funkció az **AI-alapú összefoglalás** (72%), amely már a fejlesztési tervekben szerepel.

Az eredmények megerősítik, hogy a platform használható, értékes és van létjogosultsága a piacon. A felhasználók pozitívan értékelik az egyedi funkciókat (térkép, településekre fókuszálás), és készek ajánlani azt másoknak.

7. SWOT ANALÍZIS

A SWOT analízis egy stratégiai tervezési eszköz, amely segít azonosítani egy projekt erősségeit (Strengths), gyengeségeit (Weaknesses), lehetőségeit (Opportunities) és veszélyeit (Threats). Az alábbiakban részletesen elemezzük a HistóriaWeb platform jelenlegi helyzetét ebből a szempontból.

Erősségek (Strengths)

A HistóriaWeb platform számos erősséggel rendelkezik, amelyek versenyképessé teszik a piacon és értékessé a felhasználók számára.

Egyedi piaci pozíció:

Jelenleg nincs más hasonló, kifejezetten magyar településekre fókuszáló történetmegosztó platform Magyarországon. Ez úttörő előnyt biztosít számunkra és lehetővé teszi, hogy piacvezető pozíciót alakítsunk ki.

Modern technológiai stack:

Az Angular, Node.js és React Native használata biztosítja, hogy a platform skálázható, karbantartható és bővíthető legyen. A modern keretrendszerek széles fejlesztői közösséggel rendelkeznek, ami megkönnyíti a jövőbeli fejlesztést.

Átfogó funkcionalitás:

A platform teljes körű megoldást nyújt: regisztráció, történetek írása, böngészése, értékelése, hozzászólások, képfeltöltés és mobilalkalmazás. A felhasználóknak nem kell több platformot használniuk.

Interaktív térkép:

A Magyarország-térkép vizuális és intuitív módot biztosít a települések böngészésére, ami egyedi felhasználói élményt nyújt és megkülönböztet minket más tartalom-megosztó platformoktól.

Multi-platform jelenlét:

A webes és mobilalkalmazás kombinációja széles felhasználói bázist ér el. A felhasználók bármilyen eszközről hozzáférhetnek a tartalomhoz.

Teljes településlefedés:

Az összes magyar település támogatása biztosítja, hogy a platform használható legyen bárhol az országban, még a legkisebb falvakban is.

Gyengeségek (Weaknesses)

Minden projektnek vannak gyengeségei, amelyeket fontos azonosítani és kezelni.

Korlátozott kezdeti tartalom:

Új platformként kezdetben kevés tartalom áll rendelkezésre, ami csökkenti a felhasználók érdeklődését. Kevés tartalom kevés felhasználót vonz, kevés felhasználó kevés tartalmat hoz létre.

Limitált marketing költségvetés:

Mint független projekt, nincs jelentős marketing költségvetésünk, ami megnehezíti a platform népszerűsítését és a felhasználói bázis gyors növekedését.

Kétfős fejlesztői csapat:

A korlátozott humánerőforrás lassítja a fejlesztést és a hibajavítást. Nehéz párhuzamosan több funkción dolgozni, és nincs 24/7 támogatás.

Nincs beépített moderáció:

Jelenleg nincs automatikus vagy manuális moderációs rendszer, ami problémát okozhat, ha nem megfelelő tartalom jelenik meg. A platform méretének növekedésével ez egyre nagyobb kihívást jelenthet.

Csak magyar nyelv támogatott:

A platform jelenleg kizárólag magyar nyelvű, ami korlátozza a nemzetközi felhasználók elérését és a turisták körében való használatot.

Nincs bevételi modell:

Jelenleg nincs kidolgozott üzleti modell vagy bevételi forrás, ami hosszú távon fenntarthatósági problémákhoz vezethet a szerver- és fejlesztési költségek fedezésében.

Lehetőségek (Opportunities)

A piaci és technológiai környezet számos lehetőséget kínál a platform számára.

Növekvő érdeklődés a helyi történetek iránt:

A pandémia óta megnőtt az érdeklődés a belföldi turizmus és a helyi kultúra iránt. Az emberek jobban értékelik saját környezetük történeteit és kíváncsiak a közeli települések látnivalóira.

Oktatási integrációs lehetőségek:

Az iskolák keresik a digitális eszközöket, amelyekkel a diákok kreatívan foglalkozhatnak a helytörténettel. A platform ideális projektmunkákhoz és helytörténeti kutatásokhoz.

AI technológia integrációja:

A magyar BERT és más AI technológiák integrálása jelentős versenyelőnyt biztosíthat. Az automatikus összefoglalás, helyesírás-ellenőrzés és átfogalmazás funkciókat kevés platform kínál magyar nyelven.

Turisztikai partnerségek:

Együttműködések kialakítása turisztikai irodákkal, szállodákkal és önkormányzatokkal jelentős forgalomnövekedést eredményezhet. Ezek a szervezetek érdekelt a helyi látnivalók népszerűsítésében.

Gamifikáció és közösségépítés:

A tervezett gamifikációs elemek (pontrendszer, kitüntetések) növelhetik a felhasználói elkötelezettséget és aktivitást. A közösségi funkciók bővítése erősítheti a platformhoz való ragaszkodást.

Nemzetközi kiterjesztés:

A modell sikeresen alkalmazható más országokban is. A szomszédos országok (Románia, Szlovákia, Szerbia) hasonló igényekkel rendelkezhetnek.

Pályázati lehetőségek:

Kulturális, oktatási és digitalizációs pályázatok forrást biztosíthatnak a fejlesztéshez. Az EU és a magyar kormány is támogatja a kulturális örökség digitális megőrzését.

Veszélyek (Threats)

Számos külső tényező veszélyeztetheti a projekt sikerét.

Nagy közösségi platformok versenye:

A Facebook csoportok és oldal már most is népszerű helyek helyi történetek megosztására. Ha a Facebook vagy más nagy platform hasonló funkciót vezet be, elveszíthetjük potenciális felhasználóinkat.

Tartalom minőségi problémák:

Rossz minőségű, pontatlan vagy nem megfelelő tartalom megjelenése ronthatja a platform hírnevét. A felhasználók elveszíthetik a bizalmat, ha túl sok hibás információt találnak.

Jogi kihívások:

Szerzői jogi problémák merülhetnek fel, ha felhasználók nem saját képeket vagy szövegeket töltenek fel. Rágalmazási vagy magánéletet sértő tartalmak jogi következményekkel járhatnak.

Fenntarthatósági kihívások:

A szerver- és fejlesztési költségek növekednek a felhasználószám növekedésével. Bevételi forrás nélkül nehéz lehet hosszú távon fenntartani a platformot.

Technológiai elavulás:

A használt keretrendszerek és technológiák gyorsan fejlődnek. Ha nem tartjuk naprakészen a platform technológiáját, elavulttá válhat és nehezebben lesz karbantartható.

Alacsony felhasználói aktivitás:

Ha nem sikerül elérni a kritikus tömeget, a platform 'kihalt' lesz, ami elveszi a motivációt az új felhasználóktól a csatlakozásra. Az inaktív platform nehezen élesztheto újra.

Kiberbiztonság veszélyek:

Hackelés, adatszivárgás vagy DDoS támadások súlyosan károsíthatják a platform működését és hírnevét. A felhasználók érzékeny adatainak védelme kritikus fontosságú.

Algoritmusváltozások:

A keresőmotorok algoritmusának változásai befolyásolhatják a platform láthatóságát.

SWOT mátrix összegzése

A SWOT analízis alapján látható, hogy a HistóriaWeb erős alapokkal rendelkezik (egyedi pozíció, modern technológia), de szembe kell néznie kihívásokkal is (korlátozott erőforrások, kezdeti tartalom hiánya). A legfontosabb stratégiai irányok:

- Kihasználni az erősségeket a lehetőségek kiaknázására (például AI integráció egyedi értékajánlatként)
- Kezelni a gyengeségeket, hogy csökkentsük a veszélyeket (például moderációs rendszer bevezetése)
- Partnerségek és együttműködések keresése az erőforrás-hiány kompenzálására
- Bevételi modell kidolgozása a hosszú távú fenntarthatóság biztosítására

8. TOVÁBBFEJLESZTÉSI LEHETŐSÉGEK

A HistóriaWeb platform jelenlegi állapota stabil és teljes funkcionalitással rendelkezik, azonban számos lehetőség kínálkozik a további fejlesztésre. A legizgalmasabb terület az mesterséges intelligencia integrációja, amely jelentősen javíthatja a felhasználói élményt és a platform hasznosságát.

AI integráció – Magyar BERT modell

A legfontosabb tervezett fejlesztés egy magyar nyelvű BERT (Bidirectional Encoder Representations from Transformers) modell integrálása a platformba. A BERT egy előre betanított természetes nyelvfeldolgozó (NLP) modell, amely képes megérteni a magyar nyelv kontextusát és jelentését.

Történetek összefoglalása

A magyar BERT modell egyik alkalmazása a történetek automatikus összefoglalása lenne. Sok történet hosszú és részletes, ami megnehezítheti a gyors áttekintést. Az AI képes lenne:

- Kérésre automatikusan létrehozni egy rövid, 2-3 mondatos összefoglalót
- Kijelölni a történet legfontosabb pontjait
- Különböző hosszúságú összefoglalókat generálni (rövid, közepes, részletes)
- A történet lényegét megőrizve tömöríteni a szöveget

Ez a funkció különösen hasznos lenne mobilalkalmazásban, ahol a felhasználók gyorsan szeretnének tájékozódni a történetekről anélkül, hogy el kellene olvasniuk a teljes szöveget.

Helyesírási hibák javítása

A magyar nyelvtani szabályok összetettek, és könnyen előfordulhatnak helyesírási hibák a történetek írásakor. A BERT modell képes lenne:

- Automatikusan felismerni a helyesírási hibákat
- Javaslatokat tenni a helyes írásmódra
- Kontextus alapján kiválasztani a megfelelő szót (például: 'láda' vs 'látta')
- Felismerni a gyakori gépelési hibákat
- Ékezetes karakterek helyes használatát ellenőrizni

A helyesírás-ellenőrző nem csak a publikálás előtt segítene, hanem valós időben is jelezhetne hibákat írás közben, hasonlóan a modern szövegszerkesztőkhöz.

Történetek átfogalmazása

Az AI segítségével a felhasználók átfogalmaztathatnák történeteiket különböző stílusokban és hangnemekben:

- Formális → Informális stílus (vagy fordítva)
- Egyszerűsítés gyermekek számára
- Irodalmi stílusú átfogalmazás
- Hírstílusú átírás
- Hosszú mondatok felosztása, olvashatóság javítása

Ez a funkció segíthetne abban, hogy a történetek szélesebb közönség számára elérhetőek és érthetőek legyenek. Például egy bonyolult, tudományos leírású történetet át lehetne fogalmazni közérthető nyelvre.

További AI funkciók

A BERT modellen túl további AI-alapú funkciók is tervezhetők:

- Automatikus címkézés és kategorizálás történetekhez
- Hasonló történetek ajánlása
- Képfelismerés és automatikus leírás generálás
- Chatbot asszisztens történetek kereséséhez
- Hangalapú történetbevitel transzkripcióval
- Automatikus szövegszűrés

Az AI integráció jelentősen megnövelné a HistóriaWeb hasznosságát és versenyképességét, miközben megkönnyítené a felhasználók számára a minőségi tartalom létrehozását.

További funkciók

- Szövegszerkesztési lehetőségek
- Offline történethozzáférés a mobilapplikációban

9. MIT ADOTT NEKÜNK EZ A PROJEKT

A HistóriaWeb fejlesztése nem csupán egy technikai projekt volt számunkra, hanem egy átfogó tanulási élmény, amely mind szakmailag, mind személyesen fejlődni segített minket.

Technikai készségek fejlődése

A projekt során mélyreható gyakorlati tapasztalatot szereztünk modern webfejlesztési technológiákban. Az Angular keretrendszer használata megtanított minket a komponensalapú architektúra előnyeire és a single-page application (SPA) fejlesztés fortélyaira. A TypeScript nyelvvel való munka során megtapasztaltuk a típusbiztonság értékét és azt, hogyan csökkenti ez a hibák számát.

A backend fejlesztés terén Node.js és Express.js használatával értékes tapasztalatot szereztünk a RESTful API tervezésben, az adatbázis-kezelésben és a szerveroldali biztonság implementálásában. Megértettük, hogyan kell hatékonyan strukturálni egy backend alkalmazást, és hogyan lehet skálázható megoldásokat létrehozni.

Projektmenedzsment kompetenciák

Az agilis módszertan gyakorlati alkalmazása során megtanultuk, hogyan kell hatékonyan tervezni és ütemezni a fejlesztési feladatokat. A kétheti sprintek és a rendszeres megbeszélések segítették fenntartani a projekt lendületét és időben azonosítani a problémákat.

A Trello használata során elsajátítottuk a feladatok vizuális menedzselését és a prioritások helyes meghatározását. Megtanultuk, hogyan lehet reálisan becsülni a feladatok időigényét, és hogyan lehet kezelni a váratlan akadályokat.

Csapatmunka és kommunikáció

A kétfős csapatban való munka megtanított minket a hatékony kommunikáció fontosságára. A rendszeres egyeztetések, a kód elemzések és a közös problémamegoldás során fejlődött a

szakmai kommunikációs készségünk. Megtanultuk, hogyan lehet konstruktív kritikát adni és fogadni.

A GitHub használata során elsajátítottuk a verziókezelés csapatmunkában való alkalmazását.

Problémamegoldó képesség

A fejlesztés során számtalan technikai kihívással találkoztunk, amelyek megoldása fejlesztette problémamegoldó készségeinket. Megtanultuk, hogyan kell szisztematikusan megközelíteni egy problémát, hogyan kell dokumentációt olvasni, és hogyan lehet hatékonyan keresni megoldásokat az interneten.

A debugolás és a hibakeresés során kialakult egy módszeres megközelítés, amely segít azonosítani a problémák gyökerét és nem csak a tüneteket kezelni.

Dokumentációs készségek

A projekt dokumentálása során megtanultuk, hogyan kell átlátható és hasznos dokumentációt készíteni. Megértettük, hogy a jó dokumentáció mennyire megkönnyíti a későbbi karbantartást és a csapattagok közötti tudásátadást.

Teljes életciklus tapasztalata

A projekt során végigmentünk a szoftverfejlesztés teljes életciklusán: a tervezéstől a fejlesztésen és tesztelésen át a telepítésig. Ez az átfogó tapasztalat felbecsülhetetlen értékű, mert nem csak az egyes fázisokat ismertük meg, hanem azok összefüggéseit is.

Láttuk, hogyan hatnak a korai tervezési döntések a későbbi implementációra, és mennyire fontos a megfelelő architektúra kialakítása. Megtapasztaltuk a tesztelés fontosságát és azt, hogy miért érdemes már a fejlesztés kezdetétől teszteket írni.

Önbizalom és motiváció

Talán a legfontosabb, amit a projekt adott nekünk, az az önbizalom, hogy képesek vagyunk egy komplex, valós alkalmazást elejétől a végéig elkészíteni. Látni, ahogy a kezdeti ötletből fokozatosan kialakuló működő platform, rendkívül motiváló volt.

Ez a tapasztalat megerősített minket abban, hogy a szoftverfejlesztés valóban az a terület, amelyben a jövőnket szeretnénk építeni.

10. MIT VÁRUNK EL A PROJEKTTŐL

A HistóriaWeb projekt befejezése nem jelenti a munka végét, hanem inkább egy új fejezet kezdetét. Konkrét elvárásaink és céljaink vannak a platform jövőjével kapcsolatban.

Felhasználói bázis építése

Elsődleges célunk, hogy a platform valódi felhasználói közösséget építsen ki. Reményeink szerint az első évben legalább 500-1000 regisztrált felhasználó csatlakozik, akik aktívan hozzájárulnak a tartalom létrehozásához. Szeretnénk, ha a platform minden magyar megyében lenne aktív felhasználó, aki helyi történeteket oszt meg.

Hosszú távon azt várjuk, hogy a HistóriaWeb Magyarország vezető platformjává váljon a helyi történetek megosztásában, és több tízezer történet gyűljön össze az adatbázisban.

Oktatási integráció

Elvárjuk, hogy a platform bekerüljön az oktatási szférába. Szeretnénk, ha iskolák helytörténeti projektjeik során használnák a HistóriaWeb-et, és diákok dokumentálnák saját településeik történeteit. Ez nemcsak tartalomnövekedést eredményezne, hanem nevelné a fiatalok kulturális tudatosságát is.

Tervezzük, hogy kapcsolatba lépünk oktatási intézményekkel és tanárokkal, hogy bemutassuk a platform oktatási potenciálját.

Kulturális hatás

Elvárjuk, hogy a platform hozzájáruljon a magyar kulturális örökség digitális megőrzéséhez. Szeretnénk, ha olyan történetek is dokumentálásra kerülnének, amelyek máskülönben elvesznének. Különösen fontosnak tartjuk az idősebbek tudásának megőrzését, akik még emlékeznek olyan eseményekre és helyekre, amelyekről már nincs írásos dokumentáció.

Reméljük, hogy a platform katalizátorként szolgál helyi közösségek megerősítéséhez és a lokális identitás ápolásához.

Turisztikai szerepvállalás

Elvárjuk, hogy a HistóriaWeb hasznos eszközévé váljon a belföldi turizmusnak. Szeretnénk, ha turisták a platform segítségével fedeznék fel a kevésbé ismert településeket és látnivalókat. Reményeink szerint partnerségeket tudunk kialakítani turisztikai szervezetekkel és szálláshelyekkel.

Hosszú távon az a célunk, hogy a platform ne csak magyar, hanem nemzetközi turisták számára is értékes információforrás legyen, ezért tervezzük a többnyelvű támogatást.

Technológiai fejlődés

Elvárjuk, hogy a projekt folyamatosan fejlődjön technológiai szempontból is. A tervezett AI integráció jelentős mérföldkő lesz, amely megkülönbözteti a HistóriaWeb-et más hasonló platformoktól. Szeretnénk mindig naprakészek lenni a legújabb webfejlesztési trendekkel és technológiákkal.

Fenntarthatóság

Elvárjuk, hogy a platform hosszú távon fenntartható legyen mind technikai, mind pénzügyi szempontból. Tervezzük különböző bevételi modellek kidolgozását (prémium funkciók, támogatások, partnerségek), amelyek biztosítják a működési költségek fedezését és a továbbfejlesztési források rendelkezésre állását.

Szakmai elismerés

Reméljük, hogy a projekt szakmai elismerést szerez, és jó referenciának bizonyul a karrierünk során. Szeretnénk, ha a HistóriaWeb bizonyítéka lenne annak, hogy képesek vagyunk egy komplex, valós problémát megoldó alkalmazást létrehozni.

Társadalmi hatás

Legfőbb elvárásunk, hogy a HistóriaWeb pozitív társadalmi hatást gyakoroljon. Szeretnénk, ha az emberek közelebb kerülnének saját gyökereikhez, jobban megismernék lakóhelyük történelmét, és büszkék lennének helyi közösségükre.

Reméljük, hogy a platform hozzájárul a generációk közötti párbeszédhez, amikor fiatalok dokumentálják az idősebbek történeteit. Ez nem csak kulturális értéket teremt, hanem erősíti a családi és közösségi kapcsolatokat is.

11. KÖSZÖNETNYILVÁNÍTÁS

A HistóriaWeb projekt megvalósítása során sokak támogatására és segítségére számíthattunk, akik hozzájárultak a projekt sikeréhez.

Szeretnénk köszönetet mondani minden tesztfelhasználónak, aki időt és energiát áldozott arra, hogy kipróbálja a platformot és őszinte visszajelzéseket adjon. Az ő meglátásaik nélkül nem tudtuk volna olyan felhasználóbarát felületet kialakítani, amilyen most létezik.

Hálásak vagyunk a nyílt forráskódú közösségnek, amely által a használt technológiák és eszközök szabadon elérhetőek voltak számunkra. Az Angular, Node.js, React Native és az összes felhasznált könyvtár fejlesztői nélkül ez a projekt nem valósulhatott volna meg.

Külön köszönettel tartozunk oktatóinknak, akik szakmai tudásukkal és tapasztalatukkal támogatták munkánkat a projekt megvalósítása során.

Berki Balázs tanár úrnak szeretnénk megköszönni a mobilalkalmazás-fejlesztés területén nyújtott segítségét. A React Native keretrendszer használatában és a cross-platform fejlesztés kihívásainak kezelésében nyújtott útmutatása nélkülözhetetlen volt a mobilalkalmazás sikeres implementálásához. Tanácsai segítettek abban, hogy mind Android, mind iOS platformon professzionális eredményt érjünk el.

Lakatos Sándor tanár úrnak hálásak vagyunk a webes fejlesztés területén kapott támogatásért. Az Angular keretrendszer és a Node.js backend architektúra megértésében, valamint a frontend és backend közötti hatékony kommunikáció kialakításában nyújtott szakmai iránymutatása jelentősen hozzájárult a platform technikai minőségéhez. Gyakorlati tanácsai a kód strukturálásában és az optimalizálásban felbecsülhetetlen értékűek voltak.

Németh Péterné Petrikán Valéria tanárnőnek köszönjük a szakmai angol nyelvi felkészítést, amely elengedhetetlen volt a nemzetközi dokumentációk olvasásához és a modern fejlesztői eszközök angol nyelvű kezelőfelületeinek magabiztos használatához. Az angol nyelvű szakirodalom megértése és a technikai kifejezések helyes használata nagyban segítette a projekt során felmerülő problémák megoldását és a naprakész technológiai trendek követését.

Tanáraink szakmai hozzáértése és türelmes segítsége nélkül nem sikerült volna ilyen szintű projektet megvalósítanunk. Köszönjük a bizalmat és a támogatást!

ÖSSZEFOGLALÁS

A HistóriaWeb projekt egy átfogó webes és mobilalkalmazás, amely lehetővé teszi Magyarország településeihez kapcsolódó történetek gyűjtését, megosztását és megőrzését. A fejlesztés során sikeresen implementáltunk egy modern, skálázható platformot, amely Angular, Node.js, Express.js és React Native technológiákat használ.

A projekt során mélyreható tapasztalatot szereztünk a teljes szoftverfejlesztési életciklusban, a tervezéstől a telepítésig. A kétfős csapat hatékony munkamegosztással és agilis módszertannal dolgozott, ami lehetővé tette a projekt időben való befejezését.

A platform jelenlegi állapota stabil és teljes funkcionalitással rendelkezik. A 3155 magyar település támogatott, és a felhasználók regisztrálhatnak, történeteket oszthatnak meg, értékelhetnek és hozzászólhatnak. A reszponzív dizájn biztosítja, hogy az alkalmazás minden eszközön megfelelően működjön.

A jövőben tervezzük magyar BERT modell integrálását, amely lehetővé teszi a történetek automatikus összefoglalását, helyesírás-ellenőrzését és átfogalmazását. Ez jelentősen növelné a platform hasznosságát és megkülönböztetné más hasonló szolgáltatásoktól.

A HistóriaWeb nem csupán egy technológiai projekt, hanem egy küldetés a magyar kulturális örökség digitális megőrzésére. Reményeink szerint a platform hozzájárul a helyi közösségek megerősítéséhez, az oktatás támogatásához és a belföldi turizmus fejlődéséhez.

A projekt során szerzett tapasztalatok és tudás felbecsülhetetlen értékű a szakmai pályafutásunk szempontjából. Büszkék vagyunk arra, amit elértünk, és várjuk a lehetőséget, hogy tovább fejlesszük és bővítsük a platformot a felhasználók igényei szerint.

Irodalomjegyzék és források

Angular - Google LLC. Angular fejlesztői dokumentáció.
Elérhető: https://angular.io/docs

Node.js - OpenJS Foundation. Node.js fejlesztői dokumentáció.
Elérhető: https://nodejs.org/docs/latest/api

Express.js - OpenJS Foundation. Express.js fejlesztői dokumentáció.
Elérhető: https://expressjs.com

TypeScript - Microsoft Corporation. TypeScript fejlesztői dokumentáció.
Elérhető: https://www.typescriptlang.org/docs

MySQL - Oracle Corporation. MySQL fejlesztői dokumentáció.
Elérhető: https://dev.mysql.com/doc

Bootstrap - Bootstrap Team. Bootstrap CSS keretrendszer dokumentációja.
Elérhető: https://getbootstrap.com/docs

React Native - Meta Platforms Inc. React Native fejlesztői dokumentáció.
Elérhető: https://reactnative.dev/docs

Gvwyn. Magyar vármegyék és települések listája [GitHub repository].
Elérhető: https://github.com/Gvwyn/varmegyek

Geoapify - Geoapify GmbH. Geokódolási és térképszolgáltatás.
Elérhető: https://geoapify.com

Zscout370. File:HU counties blank.svg. Wikimedia Commons. Licenc: Creative Commons Attribution-Share Alike 3.0.
Elérhető: https://commons.wikimedia.org/wiki/File:HU_counties_blank.svg

Jasmine - Pivotal Labs. Jasmine tesztelési keretrendszer dokumentációja.
Elérhető: https://jasmine.github.io

Jest - Meta Platforms Inc. Jest JavaScript tesztelési keretrendszer dokumentációja.
Elérhető: https://jestjs.io/docs

Figma - Figma Inc. Figma UI tervező eszköz súgódokumentációja.
Elérhető: https://help.figma.com

GitHub - GitHub Inc. GitHub verziókezelő platform dokumentációja.
Elérhető: https://docs.github.com

Trello - Atlassian. Trello projektmenedzsment eszköz útmutatója.
Elérhető: https://trello.com/guide

Ábrajegyzék

1. ábra: A weboldal főoldala – Magyarország interaktív megyetérképe a bejelölt településekkel

2. ábra: A regisztrációs űrlap

3. ábra: Történetek keresése és böngészése megye és település szerint

4. ábra: A történet megosztás oldal űrlapja

5. ábra: Az adatbázis táblaszerkete

6. ábra: Az Angular alkalmazás útvonalkonfigurációja

7. ábra: TypeScript komponens kódrészlet

8. ábra: A projekt GitHub repository struktúrája

9. ábra: A mobilalkalmazás főoldala

10. ábra: A projekt Trello táblája (feladatok állapota: To Do, In Progress, Testing, Done)